

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

CLASSICAL 8051-01

Architecture, Memory & Assembly Fundamentals

Architecture MCS-51 classique • Mémoire • SFR • Jeu d'instructions • Assembleur

Classic MCS-51 architecture • Memory • SFRs • Instruction set • Assembly programming

100% Assembly — no C examples • Français / English

Scope / Portée

Ce premier volume établit la base de toute la série Classical 8051. Il traite le cœur 8051/80C51 classique, son modèle mémoire, ses registres, ses SFR, l'adressage bit, les modes d'adressage, le stack, le cycle d'instruction et le jeu d'instructions Assembly. Les périphériques avancés des dérivés NXP P89LPC9xx seront traités dans les volumes suivants. / This first volume establishes the foundation for the entire Classical 8051 series: classic 8051/80C51 core, memory model, registers, SFRs, bit addressing, addressing modes, stack, instruction timing, and Assembly instruction set. Advanced NXP P89LPC9xx peripherals are reserved for later volumes.

Règle de la série / Series rule: ALL executable examples are 8051 Assembly. C is intentionally excluded.

Chaîne d'apprentissage / Learning chain: RESET → REGISTERS → MEMORY SPACES → SFR/BITS → ADDRESSING → INSTRUCTIONS → STACK/CALLS → TABLES → TIMING → DEBUG.

1. 8051 identity and CPU architecture / Identité et architecture CPU

Q1 — FR : Qu'est-ce que `8051` ?

Réponse FR : `8051` : microcontrôleur 8 bits de la famille MCS-51 avec espaces mémoire distincts, SFR et adressage bit.

Q1 — EN: What is `8051`?

Answer EN: `8051`: 8-bit MCS-51-family microcontroller with distinct memory spaces, SFRs, and bit addressing.

Q2 — FR : Pourquoi `8051` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q2 — EN: Why is `8051` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q3 — FR : Quel piège faut-il éviter avec `8051` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q3 — EN: What pitfall should be avoided with `8051`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q4 — FR : Comment vérifier `8051` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

flags avec le listing assembleur.

Q4 — EN: How would you verify `8051` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q5 — FR : Qu'est-ce que `80C51` ?

Réponse FR : `80C51` : variante CMOS de l'architecture 8051 classique, conservant le modèle de programmation MCS-51.

Q5 — EN: What is `80C51`?

Answer EN: `80C51`: CMOS implementation of the classic 8051 architecture retaining the MCS-51 programming model.

Q6 — FR : Pourquoi `80C51` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q6 — EN: Why is `80C51` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q7 — FR : Quel piège faut-il éviter avec `80C51` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q7 — EN: What pitfall should be avoided with `80C51`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q8 — FR : Comment vérifier `80C51` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q8 — EN: How would you verify `80C51` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q9 — FR : Qu'est-ce que `Accumulator A` ?

Réponse FR : `Accumulator A` : registre central 8 bits utilisé implicitement ou explicitement par de nombreuses opérations arithmétiques, logiques et de transfert.

Q9 — EN: What is `Accumulator A`?

Answer EN: `Accumulator A`: central 8-bit register used implicitly or explicitly by many arithmetic, logical, and transfer operations.

Q10 — FR : Pourquoi `Accumulator A` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q10 — EN: Why is `Accumulator A` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q11 — FR : Quel piège faut-il éviter avec `Accumulator A` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q11 — EN: What pitfall should be avoided with `Accumulator A`?**

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q12 — FR : Comment vérifier `Accumulator A` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q12 — EN: How would you verify `Accumulator A` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q13 — FR : Qu'est-ce que `B register` ?

Réponse FR : `B register` : registre 8 bits utilisé notamment par MUL AB et DIV AB et disponible comme SFR général lorsque ces instructions ne l'emploient pas.

Q13 — EN: What is `B register`?

Answer EN: `B register`: 8-bit register used notably by MUL AB and DIV AB and otherwise usable as an SFR.

Q14 — FR : Pourquoi `B register` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q14 — EN: Why is `B register` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q15 — FR : Quel piège faut-il éviter avec `B register` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q15 — EN: What pitfall should be avoided with `B register`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q16 — FR : Comment vérifier `B register` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q16 — EN: How would you verify `B register` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q17 — FR : Qu'est-ce que `PSW` ?

Réponse FR : `PSW` : Program Status Word contenant flags CPU et sélection de bank de registres.

Q17 — EN: What is `PSW`?

Answer EN: `PSW`: Program Status Word containing CPU flags and register-bank selection bits.

Q18 — FR : Pourquoi `PSW` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q18 — EN: Why is `PSW` important in 8051 Assembly?



Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q19 — FR : Quel piège faut-il éviter avec `PSW` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q19 — EN: What pitfall should be avoided with `PSW`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q20 — FR : Comment vérifier `PSW` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q20 — EN: How would you verify `PSW` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q21 — FR : Qu'est-ce que `DPTR` ?

Réponse FR : `DPTR` : pointeur de données 16 bits, composé classiquement de DPH:DPL, utilisé avec MOVX, MOVC et JMP @A+DPTR.

Q21 — EN: What is `DPTR`?

Answer EN: `DPTR`: 16-bit data pointer, classically DPH:DPL, used with MOVX, MOVC, and JMP @A+DPTR.

Q22 — FR : Pourquoi `DPTR` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q22 — EN: Why is `DPTR` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q23 — FR : Quel piège faut-il éviter avec `DPTR` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q23 — EN: What pitfall should be avoided with `DPTR`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q24 — FR : Comment vérifier `DPTR` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q24 — EN: How would you verify `DPTR` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q25 — FR : Qu'est-ce que `PC` ?

Réponse FR : `PC` : Program Counter 16 bits qui adresse la prochaine instruction dans l'espace CODE.

Q25 — EN: What is `PC`?

Answer EN: `PC`: 16-bit Program Counter addressing the next instruction in CODE space.

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q26 — FR : Pourquoi `PC` est-il important en Assembly 8051 ?**

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q26 — EN: Why is `PC` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q27 — FR : Quel piège faut-il éviter avec `PC` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q27 — EN: What pitfall should be avoided with `PC`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q28 — FR : Comment vérifier `PC` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q28 — EN: How would you verify `PC` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q29 — FR : Qu'est-ce que `SP` ?

Réponse FR : `SP` : Stack Pointer 8 bits pointant le sommet de pile dans la RAM interne classique.

Q29 — EN: What is `SP`?

Answer EN: `SP`: 8-bit Stack Pointer pointing to the top of the stack in classic internal RAM.

Q30 — FR : Pourquoi `SP` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q30 — EN: Why is `SP` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q31 — FR : Quel piège faut-il éviter avec `SP` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q31 — EN: What pitfall should be avoided with `SP`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q32 — FR : Comment vérifier `SP` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q32 — EN: How would you verify `SP` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q33 — FR : Qu'est-ce que `R0-R7` ?**

Réponse FR : `R0-R7` : huit registres de travail visibles dans la bank sélectionnée par PSW.RS1:RS0.

Q33 — EN: What is `R0-R7`?

Answer EN: `R0-R7`: eight working registers visible in the bank selected by PSW.RS1:RS0.

Q34 — FR : Pourquoi `R0-R7` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q34 — EN: Why is `R0-R7` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q35 — FR : Quel piège faut-il éviter avec `R0-R7` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q35 — EN: What pitfall should be avoided with `R0-R7`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q36 — FR : Comment vérifier `R0-R7` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q36 — EN: How would you verify `R0-R7` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q37 — FR : Qu'est-ce que `register banks` ?

Réponse FR : `register banks` : quatre banks classiques de R0-R7 occupant 00h-1Fh en RAM interne.

Q37 — EN: What is `register banks`?

Answer EN: `register banks`: four classic R0-R7 banks occupying 00h-1Fh of internal RAM.

Q38 — FR : Pourquoi `register banks` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q38 — EN: Why is `register banks` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q39 — FR : Quel piège faut-il éviter avec `register banks` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q39 — EN: What pitfall should be avoided with `register banks`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q40 — FR : Comment vérifier `register banks` au débogage ?



Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q40 — EN: How would you verify `register banks` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q41 — FR : Qu'est-ce que `Carry flag CY` ?

Réponse FR : `Carry flag CY` : flag de retenue/emprunt et bit booléen central pour arithmétique multi-octets et instructions bit.

Q41 — EN: What is `Carry flag CY`?

Answer EN: `Carry flag CY`: carry/borrow flag and central Boolean bit for multi-byte arithmetic and bit instructions.

Q42 — FR : Pourquoi `Carry flag CY` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q42 — EN: Why is `Carry flag CY` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q43 — FR : Quel piège faut-il éviter avec `Carry flag CY` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q43 — EN: What pitfall should be avoided with `Carry flag CY`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q44 — FR : Comment vérifier `Carry flag CY` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q44 — EN: How would you verify `Carry flag CY` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q45 — FR : Qu'est-ce que `Auxiliary Carry AC` ?

Réponse FR : `Auxiliary Carry AC` : flag de retenue du bit 3 vers le bit 4, notamment pertinent pour BCD/DA A.

Q45 — EN: What is `Auxiliary Carry AC`?

Answer EN: `Auxiliary Carry AC`: carry from bit 3 to bit 4, notably relevant to BCD/DA A.

Q46 — FR : Pourquoi `Auxiliary Carry AC` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q46 — EN: Why is `Auxiliary Carry AC` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q47 — FR : Quel piège faut-il éviter avec `Auxiliary Carry AC` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q47 — EN: What pitfall should be avoided with `Auxiliary Carry AC`?**

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q48 — FR : Comment vérifier `Auxiliary Carry AC` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q48 — EN: How would you verify `Auxiliary Carry AC` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q49 — FR : Qu'est-ce que `Overflow OV` ?

Réponse FR : `Overflow OV` : flag indiquant un overflow signé pour certaines opérations arithmétiques.

Q49 — EN: What is `Overflow OV`?

Answer EN: `Overflow OV`: flag indicating signed overflow for relevant arithmetic operations.

Q50 — FR : Pourquoi `Overflow OV` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q50 — EN: Why is `Overflow OV` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q51 — FR : Quel piège faut-il éviter avec `Overflow OV` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q51 — EN: What pitfall should be avoided with `Overflow OV`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q52 — FR : Comment vérifier `Overflow OV` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q52 — EN: How would you verify `Overflow OV` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q53 — FR : Qu'est-ce que `Parity P` ?

Réponse FR : `Parity P` : parité de l'accumulateur mise à jour par le cœur selon l'architecture.

Q53 — EN: What is `Parity P`?

Answer EN: `Parity P`: accumulator parity flag maintained by the core according to the architecture.

Q54 — FR : Pourquoi `Parity P` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q54 — EN: Why is `Parity P` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

corresponding instruction.

Q55 — FR : Quel piège faut-il éviter avec `Parity P` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q55 — EN: What pitfall should be avoided with `Parity P`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q56 — FR : Comment vérifier `Parity P` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q56 — EN: How would you verify `Parity P` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

2. Memory model / Modèle mémoire

Q57 — FR : Qu'est-ce que `DATA` ?

Réponse FR : `DATA` : 128 octets bas internes 00h-7Fh accessibles directement et indirectement sur le modèle classique.

Q57 — EN: What is `DATA`?

Answer EN: `DATA`: lower 128 internal bytes 00h-7Fh accessible by direct and indirect addressing on the classic model.

Q58 — FR : Pourquoi `DATA` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q58 — EN: Why is `DATA` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q59 — FR : Quel piège faut-il éviter avec `DATA` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q59 — EN: What pitfall should be avoided with `DATA`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q60 — FR : Comment vérifier `DATA` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q60 — EN: How would you verify `DATA` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q61 — FR : Qu'est-ce que `IDATA` ?

Réponse FR : `IDATA` : espace interne adressé indirectement; sur les variantes à 256 octets il inclut la RAM haute 80h-FFh distincte des

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

SFR.

Q61 — EN: What is `IDATA`?

Answer EN: `IDATA`: indirectly addressed internal space; on 256-byte variants it includes upper RAM 80h-FFh distinct from SFRs.

Q62 — FR : Pourquoi `IDATA` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q62 — EN: Why is `IDATA` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q63 — FR : Quel piège faut-il éviter avec `IDATA` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q63 — EN: What pitfall should be avoided with `IDATA`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q64 — FR : Comment vérifier `IDATA` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q64 — EN: How would you verify `IDATA` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q65 — FR : Qu'est-ce que `SFR space` ?

Réponse FR : `SFR space` : zone de registres spéciaux 80h-FFh accessible par adressage direct.

Q65 — EN: What is `SFR space`?

Answer EN: `SFR space`: special-function-register region 80h-FFh accessed by direct addressing.

Q66 — FR : Pourquoi `SFR space` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q66 — EN: Why is `SFR space` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q67 — FR : Quel piège faut-il éviter avec `SFR space` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q67 — EN: What pitfall should be avoided with `SFR space`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q68 — FR : Comment vérifier `SFR space` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q68 — EN: How would you verify `SFR space` while debugging?**

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q69 — FR : Qu'est-ce que `XDATA` ?

Réponse FR : `XDATA` : espace de données externe/auxiliaire classique jusqu'à 64 KiB accédé par MOVX.

Q69 — EN: What is `XDATA`?

Answer EN: `XDATA`: classic external/auxiliary data space up to 64 KiB accessed by MOVX.

Q70 — FR : Pourquoi `XDATA` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q70 — EN: Why is `XDATA` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q71 — FR : Quel piège faut-il éviter avec `XDATA` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q71 — EN: What pitfall should be avoided with `XDATA`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q72 — FR : Comment vérifier `XDATA` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q72 — EN: How would you verify `XDATA` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q73 — FR : Qu'est-ce que `CODE` ?

Réponse FR : `CODE` : espace programme classique jusqu'à 64 KiB, exécuté par le CPU et lu comme données via MOVC.

Q73 — EN: What is `CODE`?

Answer EN: `CODE`: classic program space up to 64 KiB, executed by CPU and readable as data with MOVC.

Q74 — FR : Pourquoi `CODE` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q74 — EN: Why is `CODE` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q75 — FR : Quel piège faut-il éviter avec `CODE` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q75 — EN: What pitfall should be avoided with `CODE`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

from device-specific extensions.

Q76 — FR : Comment vérifier `CODE` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q76 — EN: How would you verify `CODE` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q77 — FR : Qu'est-ce que `bit-addressable RAM` ?

Réponse FR : `bit-addressable RAM` : zone 20h-2Fh de RAM interne classique offrant 128 bits individuellement adressables.

Q77 — EN: What is `bit-addressable RAM`?

Answer EN: `bit-addressable RAM`: classic internal RAM region 20h-2Fh providing 128 individually addressable bits.

Q78 — FR : Pourquoi `bit-addressable RAM` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q78 — EN: Why is `bit-addressable RAM` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q79 — FR : Quel piège faut-il éviter avec `bit-addressable RAM` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q79 — EN: What pitfall should be avoided with `bit-addressable RAM`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q80 — FR : Comment vérifier `bit-addressable RAM` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q80 — EN: How would you verify `bit-addressable RAM` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q81 — FR : Qu'est-ce que `bit-addressable SFR` ?

Réponse FR : `bit-addressable SFR` : certains SFR dont l'adresse de base se termine par 0h ou 8h sont bit-adressables lorsque définis ainsi par le composant.

Q81 — EN: What is `bit-addressable SFR`?

Answer EN: `bit-addressable SFR`: certain SFRs with suitable base addresses are bit-addressable when defined as such by the device.

Q82 — FR : Pourquoi `bit-addressable SFR` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q82 — EN: Why is `bit-addressable SFR` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

**Q83 — FR : Quel piège faut-il éviter avec `bit-addressable SFR` ?**

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q83 — EN: What pitfall should be avoided with `bit-addressable SFR`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q84 — FR : Comment vérifier `bit-addressable SFR` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q84 — EN: How would you verify `bit-addressable SFR` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q85 — FR : Qu'est-ce que `MOV` versus `MOVX` ?

Réponse FR : `MOV` versus `MOVX` : `MOV` travaille avec registres/RAM interne/SFR; `MOVX` cible XDATA.

Q85 — EN: What is `MOV` versus `MOVX`?

Answer EN: `MOV` versus `MOVX`: `MOV` works with registers/internal RAM/SFRs; `MOVX` targets XDATA.

Q86 — FR : Pourquoi `MOV` versus `MOVX` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q86 — EN: Why is `MOV` versus `MOVX` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q87 — FR : Quel piège faut-il éviter avec `MOV` versus `MOVX` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q87 — EN: What pitfall should be avoided with `MOV` versus `MOVX`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q88 — FR : Comment vérifier `MOV` versus `MOVX` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q88 — EN: How would you verify `MOV` versus `MOVX` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q89 — FR : Qu'est-ce que `MOVC` ?

Réponse FR : `MOVC` : lit un octet depuis CODE à une adresse formée avec A+DPTR ou A+PC.

Q89 — EN: What is `MOVC`?

Answer EN: `MOVC`: reads a byte from CODE using an address formed by A+DPTR or A+PC.

Q90 — FR : Pourquoi `MOVC` est-il important en Assembly 8051 ?



Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q90 — EN: Why is `MOVC` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q91 — FR : Quel piège faut-il éviter avec `MOVC` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q91 — EN: What pitfall should be avoided with `MOVC`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q92 — FR : Comment vérifier `MOVC` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q92 — EN: How would you verify `MOVC` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q93 — FR : Qu'est-ce que `Harvard-like separation` ?

Réponse FR : `Harvard-like separation` : CODE et DATA/XDATA ont des mécanismes d'accès distincts même si leurs valeurs d'adresse peuvent être identiques.

Q93 — EN: What is `Harvard-like separation`?

Answer EN: `Harvard-like separation`: CODE and DATA/XDATA use distinct access mechanisms even when numeric addresses match.

Q94 — FR : Pourquoi `Harvard-like separation` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q94 — EN: Why is `Harvard-like separation` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q95 — FR : Quel piège faut-il éviter avec `Harvard-like separation` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q95 — EN: What pitfall should be avoided with `Harvard-like separation`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q96 — FR : Comment vérifier `Harvard-like separation` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q96 — EN: How would you verify `Harvard-like separation` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

3. Assembly source structure / Structure d'un source Assembly

Q97 — FR : Qu'est-ce que `ORG` ?

Réponse FR : `ORG` : directive d'assemblage plaçant le compteur d'emplacement à une adresse; ce n'est pas une instruction exécutée par le CPU.

Q97 — EN: What is `ORG`?

Answer EN: `ORG`: assembler directive setting the location counter; it is not a CPU-executed instruction.

Q98 — FR : Pourquoi `ORG` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q98 — EN: Why is `ORG` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q99 — FR : Quel piège faut-il éviter avec `ORG` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q99 — EN: What pitfall should be avoided with `ORG`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q100 — FR : Comment vérifier `ORG` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q100 — EN: How would you verify `ORG` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q101 — FR : Qu'est-ce que `END` ?

Réponse FR : `END` : directive marquant classiquement la fin du module source pour l'assembleur.

Q101 — EN: What is `END`?

Answer EN: `END`: directive classically marking the end of the source module for the assembler.

Q102 — FR : Pourquoi `END` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q102 — EN: Why is `END` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q103 — FR : Quel piège faut-il éviter avec `END` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q103 — EN: What pitfall should be avoided with `END`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q104 — FR : Comment vérifier `END` au débogage ?**

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q104 — EN: How would you verify `END` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q105 — FR : Qu'est-ce que `EQU` ?

Réponse FR : `EQU` : directive donnant un nom symbolique à une constante sans réserver de RAM.

Q105 — EN: What is `EQU`?

Answer EN: `EQU`: directive assigning a symbolic name to a constant without allocating RAM.

Q106 — FR : Pourquoi `EQU` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q106 — EN: Why is `EQU` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q107 — FR : Quel piège faut-il éviter avec `EQU` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q107 — EN: What pitfall should be avoided with `EQU`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q108 — FR : Comment vérifier `EQU` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q108 — EN: How would you verify `EQU` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q109 — FR : Qu'est-ce que `DB` ?

Réponse FR : `DB` : directive plaçant des octets constants dans l'image assemblée, utile pour tables en CODE.

Q109 — EN: What is `DB`?

Answer EN: `DB`: directive placing constant bytes in the assembled image, useful for CODE tables.

Q110 — FR : Pourquoi `DB` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q110 — EN: Why is `DB` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q111 — FR : Quel piège faut-il éviter avec `DB` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email:** ahds@engineer.com

classique des extensions du composant réel.

Q111 — EN: What pitfall should be avoided with `DB`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q112 — FR : Comment vérifier `DB` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q112 — EN: How would you verify `DB` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q113 — FR : Qu'est-ce que `label` ?

Réponse FR : `label` : nom symbolique associé à une adresse de code ou de données selon le contexte.

Q113 — EN: What is `label`?

Answer EN: `label`: symbolic name associated with a code or data address depending on context.

Q114 — FR : Pourquoi `label` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q114 — EN: Why is `label` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q115 — FR : Quel piège faut-il éviter avec `label` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q115 — EN: What pitfall should be avoided with `label`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q116 — FR : Comment vérifier `label` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q116 — EN: How would you verify `label` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q117 — FR : Qu'est-ce que `reset vector` ?

Réponse FR : `reset vector` : adresse 0000h où l'exécution commence après reset sur le modèle 8051 classique.

Q117 — EN: What is `reset vector`?

Answer EN: `reset vector`: address 0000h where execution begins after reset on the classic 8051 model.

Q118 — FR : Pourquoi `reset vector` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q118 — EN: Why is `reset vector` important in 8051 Assembly?

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q119 — FR : Quel piège faut-il éviter avec `reset vector` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q119 — EN: What pitfall should be avoided with `reset vector`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q120 — FR : Comment vérifier `reset vector` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q120 — EN: How would you verify `reset vector` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q121 — FR : Qu'est-ce que `vector stub` ?

Réponse FR : `vector stub` : petit saut placé à un vecteur pour rediriger vers une routine plus éloignée.

Q121 — EN: What is `vector stub`?

Answer EN: `vector stub`: small jump placed at a vector to redirect to a more distant routine.

Q122 — FR : Pourquoi `vector stub` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q122 — EN: Why is `vector stub` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q123 — FR : Quel piège faut-il éviter avec `vector stub` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q123 — EN: What pitfall should be avoided with `vector stub`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q124 — FR : Comment vérifier `vector stub` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q124 — EN: How would you verify `vector stub` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

4. Addressing modes / Modes d'adressage

Q125 — FR : Qu'est-ce que `immediate addressing` ?

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Réponse FR : `immediate addressing` : opérande constant précédé classiquement de #, par exemple MOV A,#55h.

Q125 — EN: What is `immediate addressing`?

Answer EN: `immediate addressing`: constant operand classically prefixed by #, e.g. MOV A,#55h.

Q126 — FR : Pourquoi `immediate addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q126 — EN: Why is `immediate addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q127 — FR : Quel piège faut-il éviter avec `immediate addressing` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q127 — EN: What pitfall should be avoided with `immediate addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q128 — FR : Comment vérifier `immediate addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q128 — EN: How would you verify `immediate addressing` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q129 — FR : Qu'est-ce que `register addressing` ?

Réponse FR : `register addressing` : opérande R0-R7 de la bank active ou registre explicitement supporté.

Q129 — EN: What is `register addressing`?

Answer EN: `register addressing`: operand in R0-R7 of the active bank or another explicitly supported register.

Q130 — FR : Pourquoi `register addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q130 — EN: Why is `register addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q131 — FR : Quel piège faut-il éviter avec `register addressing` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q131 — EN: What pitfall should be avoided with `register addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q132 — FR : Comment vérifier `register addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q132 — EN: How would you verify `register addressing` while debugging?**

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q133 — FR : Qu'est-ce que `direct addressing` ?

Réponse FR : `direct addressing` : octet d'adresse inclus dans l'instruction, visant DATA basse ou SFR selon la valeur.

Q133 — EN: What is `direct addressing`?

Answer EN: `direct addressing`: address byte encoded in the instruction, targeting lower DATA or an SFR depending on value.

Q134 — FR : Pourquoi `direct addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q134 — EN: Why is `direct addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q135 — FR : Quel piège faut-il éviter avec `direct addressing` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q135 — EN: What pitfall should be avoided with `direct addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q136 — FR : Comment vérifier `direct addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q136 — EN: How would you verify `direct addressing` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q137 — FR : Qu'est-ce que `register-indirect addressing` ?

Réponse FR : `register-indirect addressing` : adresse contenue dans R0/R1 pour RAM interne ou dans DPTR/Ri selon la forme MOVX.

Q137 — EN: What is `register-indirect addressing`?

Answer EN: `register-indirect addressing`: address held in R0/R1 for internal RAM or DPTR/Ri for supported MOVX forms.

Q138 — FR : Pourquoi `register-indirect addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q138 — EN: Why is `register-indirect addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q139 — FR : Quel piège faut-il éviter avec `register-indirect addressing` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q139 — EN: What pitfall should be avoided with `register-indirect addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

from device-specific extensions.

Q140 — FR : Comment vérifier `register-indirect addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q140 — EN: How would you verify `register-indirect addressing` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q141 — FR : Qu'est-ce que `indexed CODE addressing` ?

Réponse FR : `indexed CODE addressing` : A sert d'index ajouté à DPTR ou PC pour MOVC.

Q141 — EN: What is `indexed CODE addressing`?

Answer EN: `indexed CODE addressing`: A acts as an index added to DPTR or PC for MOVC.

Q142 — FR : Pourquoi `indexed CODE addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q142 — EN: Why is `indexed CODE addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q143 — FR : Quel piège faut-il éviter avec `indexed CODE addressing` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q143 — EN: What pitfall should be avoided with `indexed CODE addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q144 — FR : Comment vérifier `indexed CODE addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q144 — EN: How would you verify `indexed CODE addressing` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q145 — FR : Qu'est-ce que `bit addressing` ?

Réponse FR : `bit addressing` : instruction visant directement un bit adressable RAM/SFR.

Q145 — EN: What is `bit addressing`?

Answer EN: `bit addressing`: instruction directly targeting an addressable RAM/SFR bit.

Q146 — FR : Pourquoi `bit addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q146 — EN: Why is `bit addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

**Q147 — FR : Quel piège faut-il éviter avec `bit addressing` ?**

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q147 — EN: What pitfall should be avoided with `bit addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q148 — FR : Comment vérifier `bit addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q148 — EN: How would you verify `bit addressing` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q149 — FR : Qu'est-ce que `relative branch` ?

Réponse FR : `relative branch` : branche utilisant un déplacement signé 8 bits relatif à l'instruction suivante.

Q149 — EN: What is `relative branch`?

Answer EN: `relative branch`: branch using a signed 8-bit displacement relative to the following instruction.

Q150 — FR : Pourquoi `relative branch` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q150 — EN: Why is `relative branch` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q151 — FR : Quel piège faut-il éviter avec `relative branch` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q151 — EN: What pitfall should be avoided with `relative branch`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q152 — FR : Comment vérifier `relative branch` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q152 — EN: How would you verify `relative branch` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q153 — FR : Qu'est-ce que `AJMP/ACALL page addressing` ?

Réponse FR : `AJMP/ACALL page addressing` : forme courte limitée à la même page de 2 KiB du point de vue de l'encodage classique.

Q153 — EN: What is `AJMP/ACALL page addressing`?

Answer EN: `AJMP/ACALL page addressing`: short form limited by classic encoding to the same 2-KiB page.

Q154 — FR : Pourquoi `AJMP/ACALL page addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

son effet avant d'écrire l'instruction correspondante.

Q154 — EN: Why is `AJMP/ACALL page addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q155 — FR : Quel piège faut-il éviter avec `AJMP/ACALL page addressing` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q155 — EN: What pitfall should be avoided with `AJMP/ACALL page addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q156 — FR : Comment vérifier `AJMP/ACALL page addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q156 — EN: How would you verify `AJMP/ACALL page addressing` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.

Q157 — FR : Qu'est-ce que `LJMP/LCALL absolute addressing` ?

Réponse FR : `LJMP/LCALL absolute addressing` : forme longue utilisant une destination CODE 16 bits.

Q157 — EN: What is `LJMP/LCALL absolute addressing`?

Answer EN: `LJMP/LCALL absolute addressing`: long form using a 16-bit CODE destination.

Q158 — FR : Pourquoi `LJMP/LCALL absolute addressing` est-il important en Assembly 8051 ?

Réponse FR : Parce qu'en Assembly le programmeur manipule directement ce mécanisme. Il faut connaître son adresse, sa portée ou son effet avant d'écrire l'instruction correspondante.

Q158 — EN: Why is `LJMP/LCALL absolute addressing` important in 8051 Assembly?

Answer EN: Because Assembly exposes this mechanism directly. You must know its address, scope, or effect before selecting the corresponding instruction.

Q159 — FR : Quel piège faut-il éviter avec `LJMP/LCALL absolute addressing` ?

Réponse FR : Ne pas lui attribuer les propriétés d'un autre espace mémoire ou d'un autre dérivé 8051. Toujours distinguer le cœur classique des extensions du composant réel.

Q159 — EN: What pitfall should be avoided with `LJMP/LCALL absolute addressing`?

Answer EN: Do not assign properties from another memory space or another 8051 derivative. Always separate classic-core behavior from device-specific extensions.

Q160 — FR : Comment vérifier `LJMP/LCALL absolute addressing` au débogage ?

Réponse FR : Observer les registres/mémoires concernés dans le debugger, exécuter pas à pas et comparer l'adresse effective et les flags avec le listing assembleur.

Q160 — EN: How would you verify `LJMP/LCALL absolute addressing` while debugging?

Answer EN: Inspect the relevant registers/memory in the debugger, single-step, and compare the effective address and flags with the assembly listing.



5. Complete classic instruction families / Familles d'instructions classiques

Q161 — FR : Que fait `MOV` ?

Réponse FR : `MOV` copie un octet entre les combinaisons d'opérandes supportées sans passer obligatoirement par A.

Q161 — EN: What does `MOV` do?

Answer EN: `MOV` copies one byte between supported operands without necessarily using A.

Q162 — FR : Quand utiliser `MOV` ?

Réponse FR : Utilisez `MOV` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q162 — EN: When should `MOV` be used?

Answer EN: Use `MOV` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q163 — FR : Quel détail faut-il contrôler avant `MOV` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q163 — EN: What should be checked before `MOV`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q164 — FR : Comment déboguer une erreur liée à `MOV` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q164 — EN: How do you debug an error involving `MOV`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q165 — FR : Que fait `MOVX` ?

Réponse FR : `MOVX` transfère un octet entre A et XDATA via DPTR ou R0/R1 selon la forme.

Q165 — EN: What does `MOVX` do?

Answer EN: `MOVX` transfers a byte between A and XDATA through DPTR or R0/R1 depending on form.

Q166 — FR : Quand utiliser `MOVX` ?

Réponse FR : Utilisez `MOVX` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q166 — EN: When should `MOVX` be used?

Answer EN: Use `MOVX` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q167 — FR : Quel détail faut-il contrôler avant `MOVX` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q167 — EN: What should be checked before `MOVX`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q168 — FR : Comment déboguer une erreur liée à `MOVX` ?



Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q168 — EN: How do you debug an error involving `MOVX`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q169 — FR : Que fait `MOVC` ?

Réponse FR : `MOVC` charge A depuis CODE avec index A+DPTR ou A+PC.

Q169 — EN: What does `MOVC` do?

Answer EN: `MOVC` loads A from CODE using A+DPTR or A+PC indexing.

Q170 — FR : Quand utiliser `MOVC` ?

Réponse FR : Utilisez `MOVC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q170 — EN: When should `MOVC` be used?

Answer EN: Use `MOVC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q171 — FR : Quel détail faut-il contrôler avant `MOVC` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q171 — EN: What should be checked before `MOVC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q172 — FR : Comment déboguer une erreur liée à `MOVC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q172 — EN: How do you debug an error involving `MOVC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q173 — FR : Que fait `XCH` ?

Réponse FR : `XCH` échange A avec un octet registre/direct/indirect supporté.

Q173 — EN: What does `XCH` do?

Answer EN: `XCH` exchanges A with a supported register/direct/indirect byte.

Q174 — FR : Quand utiliser `XCH` ?

Réponse FR : Utilisez `XCH` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q174 — EN: When should `XCH` be used?

Answer EN: Use `XCH` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q175 — FR : Quel détail faut-il contrôler avant `XCH` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q175 — EN: What should be checked before `XCH`?**

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q176 — FR : Comment déboguer une erreur liée à `XCH` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q176 — EN: How do you debug an error involving `XCH`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q177 — FR : Que fait `XCHD` ?

Réponse FR : `XCHD` échange les nibbles bas entre A et la RAM pointée indirectement par R0/R1.

Q177 — EN: What does `XCHD` do?

Answer EN: `XCHD` exchanges low nibbles between A and RAM indirectly addressed by R0/R1.

Q178 — FR : Quand utiliser `XCHD` ?

Réponse FR : Utilisez `XCHD` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q178 — EN: When should `XCHD` be used?

Answer EN: Use `XCHD` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q179 — FR : Quel détail faut-il contrôler avant `XCHD` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q179 — EN: What should be checked before `XCHD`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q180 — FR : Comment déboguer une erreur liée à `XCHD` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q180 — EN: How do you debug an error involving `XCHD`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q181 — FR : Que fait `PUSH` ?

Réponse FR : `PUSH` incrémente SP puis copie l'octet direct sur la pile.

Q181 — EN: What does `PUSH` do?

Answer EN: `PUSH` increments SP then copies the direct byte onto the stack.

Q182 — FR : Quand utiliser `PUSH` ?

Réponse FR : Utilisez `PUSH` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q182 — EN: When should `PUSH` be used?

Answer EN: Use `PUSH` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

in the instruction table.

Q183 — FR : Quel détail faut-il contrôler avant `PUSH` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q183 — EN: What should be checked before `PUSH`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q184 — FR : Comment déboguer une erreur liée à `PUSH` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q184 — EN: How do you debug an error involving `PUSH`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q185 — FR : Que fait `POP` ?

Réponse FR : `POP` copie le sommet de pile vers l'octet direct puis décrémente SP.

Q185 — EN: What does `POP` do?

Answer EN: `POP` copies the stack top to a direct byte then decrements SP.

Q186 — FR : Quand utiliser `POP` ?

Réponse FR : Utilisez `POP` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q186 — EN: When should `POP` be used?

Answer EN: Use `POP` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q187 — FR : Quel détail faut-il contrôler avant `POP` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q187 — EN: What should be checked before `POP`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q188 — FR : Comment déboguer une erreur liée à `POP` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q188 — EN: How do you debug an error involving `POP`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q189 — FR : Que fait `ADD` ?

Réponse FR : `ADD` additionne un opérande à A et met à jour les flags arithmétiques pertinents.

Q189 — EN: What does `ADD` do?

Answer EN: `ADD` adds an operand to A and updates relevant arithmetic flags.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q190 — FR : Quand utiliser `ADD` ?**

Réponse FR : Utilisez `ADD` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q190 — EN: When should `ADD` be used?

Answer EN: Use `ADD` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q191 — FR : Quel détail faut-il contrôler avant `ADD` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q191 — EN: What should be checked before `ADD`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q192 — FR : Comment déboguer une erreur liée à `ADD` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q192 — EN: How do you debug an error involving `ADD`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q193 — FR : Que fait `ADDC` ?

Réponse FR : `ADDC` additionne opérande + CY à A, essentiel pour l'arithmétique multi-octets.

Q193 — EN: What does `ADDC` do?

Answer EN: `ADDC` adds operand plus CY to A, essential for multi-byte arithmetic.

Q194 — FR : Quand utiliser `ADDC` ?

Réponse FR : Utilisez `ADDC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q194 — EN: When should `ADDC` be used?

Answer EN: Use `ADDC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q195 — FR : Quel détail faut-il contrôler avant `ADDC` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q195 — EN: What should be checked before `ADDC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q196 — FR : Comment déboguer une erreur liée à `ADDC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q196 — EN: How do you debug an error involving `ADDC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q197 — FR : Que fait `SUBB` ?**

Réponse FR : `SUBB` soustrait opérande et emprunt CY de A.

Q197 — EN: What does `SUBB` do?

Answer EN: `SUBB` subtracts operand and borrow CY from A.

Q198 — FR : Quand utiliser `SUBB` ?

Réponse FR : Utilisez `SUBB` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q198 — EN: When should `SUBB` be used?

Answer EN: Use `SUBB` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q199 — FR : Quel détail faut-il contrôler avant `SUBB` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q199 — EN: What should be checked before `SUBB`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q200 — FR : Comment déboguer une erreur liée à `SUBB` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q200 — EN: How do you debug an error involving `SUBB`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q201 — FR : Que fait `INC` ?

Réponse FR : `INC` incrémente l'opérande supporté sans utiliser CY comme retenue.

Q201 — EN: What does `INC` do?

Answer EN: `INC` increments a supported operand without using CY as carry.

Q202 — FR : Quand utiliser `INC` ?

Réponse FR : Utilisez `INC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q202 — EN: When should `INC` be used?

Answer EN: Use `INC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q203 — FR : Quel détail faut-il contrôler avant `INC` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q203 — EN: What should be checked before `INC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q204 — FR : Comment déboguer une erreur liée à `INC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le



listing pour confirmer l'encodage et la destination réelle.

Q204 — EN: How do you debug an error involving `INC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q205 — FR : Que fait `DEC` ?

Réponse FR : `DEC` décrémente l'opérande supporté.

Q205 — EN: What does `DEC` do?

Answer EN: `DEC` decrements a supported operand.

Q206 — FR : Quand utiliser `DEC` ?

Réponse FR : Utilisez `DEC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q206 — EN: When should `DEC` be used?

Answer EN: Use `DEC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q207 — FR : Quel détail faut-il contrôler avant `DEC` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q207 — EN: What should be checked before `DEC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q208 — FR : Comment déboguer une erreur liée à `DEC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q208 — EN: How do you debug an error involving `DEC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q209 — FR : Que fait `MUL AB` ?

Réponse FR : `MUL AB` multiplie A par B; résultat 16 bits réparti classiquement entre B:A.

Q209 — EN: What does `MUL AB` do?

Answer EN: `MUL AB` multiplies A by B; 16-bit result is classically returned in B:A.

Q210 — FR : Quand utiliser `MUL AB` ?

Réponse FR : Utilisez `MUL AB` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q210 — EN: When should `MUL AB` be used?

Answer EN: Use `MUL AB` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q211 — FR : Quel détail faut-il contrôler avant `MUL AB` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q211 — EN: What should be checked before `MUL AB`?

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q212 — FR : Comment déboguer une erreur liée à `MUL AB` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q212 — EN: How do you debug an error involving `MUL AB`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q213 — FR : Que fait `DIV AB` ?

Réponse FR : `DIV AB` divise A par B; quotient/reste sont placés dans A/B selon l'architecture.

Q213 — EN: What does `DIV AB` do?

Answer EN: `DIV AB` divides A by B; quotient/remainder are placed in A/B according to the architecture.

Q214 — FR : Quand utiliser `DIV AB` ?

Réponse FR : Utilisez `DIV AB` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q214 — EN: When should `DIV AB` be used?

Answer EN: Use `DIV AB` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q215 — FR : Quel détail faut-il contrôler avant `DIV AB` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q215 — EN: What should be checked before `DIV AB`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q216 — FR : Comment déboguer une erreur liée à `DIV AB` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q216 — EN: How do you debug an error involving `DIV AB`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q217 — FR : Que fait `DA A` ?

Réponse FR : `DA A` ajuste A après une addition BCD selon AC/CY et les nibbles.

Q217 — EN: What does `DA A` do?

Answer EN: `DA A` decimal-adjusts A after BCD addition according to AC/CY and nibble values.

Q218 — FR : Quand utiliser `DA A` ?

Réponse FR : Utilisez `DA A` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q218 — EN: When should `DA A` be used?

Answer EN: Use `DA A` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q219 — FR : Quel détail faut-il contrôler avant `DA A` ?**

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q219 — EN: What should be checked before `DA A` ?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q220 — FR : Comment déboguer une erreur liée à `DA A` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q220 — EN: How do you debug an error involving `DA A` ?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q221 — FR : Que fait `ANL` ?

Réponse FR : `ANL` effectue un AND logique sur les formes byte/bit supportées.

Q221 — EN: What does `ANL` do?

Answer EN: `ANL` performs logical AND on supported byte/bit forms.

Q222 — FR : Quand utiliser `ANL` ?

Réponse FR : Utilisez `ANL` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q222 — EN: When should `ANL` be used?

Answer EN: Use `ANL` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q223 — FR : Quel détail faut-il contrôler avant `ANL` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q223 — EN: What should be checked before `ANL`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q224 — FR : Comment déboguer une erreur liée à `ANL` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q224 — EN: How do you debug an error involving `ANL`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q225 — FR : Que fait `ORL` ?

Réponse FR : `ORL` effectue un OR logique sur les formes supportées.

Q225 — EN: What does `ORL` do?

Answer EN: `ORL` performs logical OR on supported forms.

Q226 — FR : Quand utiliser `ORL` ?

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Réponse FR : Utilisez `ORL` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q226 — EN: When should `ORL` be used?

Answer EN: Use `ORL` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q227 — FR : Quel détail faut-il contrôler avant `ORL` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q227 — EN: What should be checked before `ORL`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q228 — FR : Comment déboguer une erreur liée à `ORL` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q228 — EN: How do you debug an error involving `ORL`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q229 — FR : Que fait `XRL` ?

Réponse FR : `XRL` effectue un XOR logique sur les formes supportées.

Q229 — EN: What does `XRL` do?

Answer EN: `XRL` performs logical XOR on supported forms.

Q230 — FR : Quand utiliser `XRL` ?

Réponse FR : Utilisez `XRL` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q230 — EN: When should `XRL` be used?

Answer EN: Use `XRL` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q231 — FR : Quel détail faut-il contrôler avant `XRL` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q231 — EN: What should be checked before `XRL`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q232 — FR : Comment déboguer une erreur liée à `XRL` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q232 — EN: How do you debug an error involving `XRL`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q233 — FR : Que fait `CLR` ?

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

Réponse FR : `CLR` met à zéro A, CY ou un bit selon la forme.

Q233 — EN: What does `CLR` do?

Answer EN: `CLR` clears A, CY, or a bit depending on form.

Q234 — FR : Quand utiliser `CLR` ?

Réponse FR : Utilisez `CLR` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q234 — EN: When should `CLR` be used?

Answer EN: Use `CLR` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q235 — FR : Quel détail faut-il contrôler avant `CLR` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q235 — EN: What should be checked before `CLR`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q236 — FR : Comment déboguer une erreur liée à `CLR` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q236 — EN: How do you debug an error involving `CLR`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q237 — FR : Que fait `CPL` ?

Réponse FR : `CPL` complémente A, CY ou un bit selon la forme.

Q237 — EN: What does `CPL` do?

Answer EN: `CPL` complements A, CY, or a bit depending on form.

Q238 — FR : Quand utiliser `CPL` ?

Réponse FR : Utilisez `CPL` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q238 — EN: When should `CPL` be used?

Answer EN: Use `CPL` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q239 — FR : Quel détail faut-il contrôler avant `CPL` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q239 — EN: What should be checked before `CPL`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q240 — FR : Comment déboguer une erreur liée à `CPL` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q240 — EN: How do you debug an error involving `CPL`?**

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q241 — FR : Que fait `RL` ?

Réponse FR : `RL` rotation gauche de A sans CY.

Q241 — EN: What does `RL` do?

Answer EN: `RL` rotates A left without CY.

Q242 — FR : Quand utiliser `RL` ?

Réponse FR : Utilisez `RL` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q242 — EN: When should `RL` be used?

Answer EN: Use `RL` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q243 — FR : Quel détail faut-il contrôler avant `RL` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q243 — EN: What should be checked before `RL`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q244 — FR : Comment déboguer une erreur liée à `RL` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q244 — EN: How do you debug an error involving `RL`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q245 — FR : Que fait `RLC` ?

Réponse FR : `RLC` rotation gauche de A à travers CY.

Q245 — EN: What does `RLC` do?

Answer EN: `RLC` rotates A left through CY.

Q246 — FR : Quand utiliser `RLC` ?

Réponse FR : Utilisez `RLC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q246 — EN: When should `RLC` be used?

Answer EN: Use `RLC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q247 — FR : Quel détail faut-il contrôler avant `RLC` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q247 — EN: What should be checked before `RLC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

modifies.

Q248 — FR : Comment déboguer une erreur liée à `RLC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q248 — EN: How do you debug an error involving `RLC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q249 — FR : Que fait `RR` ?

Réponse FR : `RR` rotation droite de A sans CY.

Q249 — EN: What does `RR` do?

Answer EN: `RR` rotates A right without CY.

Q250 — FR : Quand utiliser `RR` ?

Réponse FR : Utilisez `RR` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q250 — EN: When should `RR` be used?

Answer EN: Use `RR` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q251 — FR : Quel détail faut-il contrôler avant `RR` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q251 — EN: What should be checked before `RR`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q252 — FR : Comment déboguer une erreur liée à `RR` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q252 — EN: How do you debug an error involving `RR`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q253 — FR : Que fait `RRC` ?

Réponse FR : `RRC` rotation droite de A à travers CY.

Q253 — EN: What does `RRC` do?

Answer EN: `RRC` rotates A right through CY.

Q254 — FR : Quand utiliser `RRC` ?

Réponse FR : Utilisez `RRC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q254 — EN: When should `RRC` be used?

Answer EN: Use `RRC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q255 — FR : Quel détail faut-il contrôler avant `RRC` ?**

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q255 — EN: What should be checked before `RRC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q256 — FR : Comment déboguer une erreur liée à `RRC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q256 — EN: How do you debug an error involving `RRC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q257 — FR : Que fait `SWAP A` ?

Réponse FR : `SWAP A` échange les nibbles haut et bas de A.

Q257 — EN: What does `SWAP A` do?

Answer EN: `SWAP A` swaps high and low nibbles of A.

Q258 — FR : Quand utiliser `SWAP A` ?

Réponse FR : Utilisez `SWAP A` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q258 — EN: When should `SWAP A` be used?

Answer EN: Use `SWAP A` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q259 — FR : Quel détail faut-il contrôler avant `SWAP A` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q259 — EN: What should be checked before `SWAP A`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q260 — FR : Comment déboguer une erreur liée à `SWAP A` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q260 — EN: How do you debug an error involving `SWAP A`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q261 — FR : Que fait `SETB` ?

Réponse FR : `SETB` met CY ou un bit adressable à 1.

Q261 — EN: What does `SETB` do?

Answer EN: `SETB` sets CY or an addressable bit.

Q262 — FR : Quand utiliser `SETB` ?

Réponse FR : Utilisez `SETB` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q262 — EN: When should `SETB` be used?

Answer EN: Use `SETB` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q263 — FR : Quel détail faut-il contrôler avant `SETB` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q263 — EN: What should be checked before `SETB` ?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q264 — FR : Comment déboguer une erreur liée à `SETB` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q264 — EN: How do you debug an error involving `SETB` ?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q265 — FR : Que fait `SJMP` ?

Réponse FR : `SJMP` saut relatif inconditionnel.

Q265 — EN: What does `SJMP` do?

Answer EN: `SJMP` unconditional relative jump.

Q266 — FR : Quand utiliser `SJMP` ?

Réponse FR : Utilisez `SJMP` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q266 — EN: When should `SJMP` be used?

Answer EN: Use `SJMP` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q267 — FR : Quel détail faut-il contrôler avant `SJMP` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q267 — EN: What should be checked before `SJMP`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q268 — FR : Comment déboguer une erreur liée à `SJMP` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q268 — EN: How do you debug an error involving `SJMP`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q269 — FR : Que fait `AJMP` ?

Réponse FR : `AJMP` saut absolu court dans la page d'encodage classique.

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q269 — EN: What does `AJMP` do?**

Answer EN: `AJMP` short absolute jump within the classic encoding page.

Q270 — FR : Quand utiliser `AJMP` ?

Réponse FR : Utilisez `AJMP` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q270 — EN: When should `AJMP` be used?

Answer EN: Use `AJMP` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q271 — FR : Quel détail faut-il contrôler avant `AJMP` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q271 — EN: What should be checked before `AJMP`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q272 — FR : Comment déboguer une erreur liée à `AJMP` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q272 — EN: How do you debug an error involving `AJMP`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q273 — FR : Que fait `LJMP` ?

Réponse FR : `LJMP` saut inconditionnel vers une adresse CODE 16 bits.

Q273 — EN: What does `LJMP` do?

Answer EN: `LJMP` unconditional jump to a 16-bit CODE address.

Q274 — FR : Quand utiliser `LJMP` ?

Réponse FR : Utilisez `LJMP` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q274 — EN: When should `LJMP` be used?

Answer EN: Use `LJMP` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q275 — FR : Quel détail faut-il contrôler avant `LJMP` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q275 — EN: What should be checked before `LJMP`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q276 — FR : Comment déboguer une erreur liée à `LJMP` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q276 — EN: How do you debug an error involving `LJMP`?



Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q277 — FR : Que fait `JMP @A+DPTR` ?

Réponse FR : `JMP @A+DPTR` saut indirect indexé, utile pour tables de dispatch.

Q277 — EN: What does `JMP @A+DPTR` do?

Answer EN: `JMP @A+DPTR` indexed indirect jump useful for dispatch tables.

Q278 — FR : Quand utiliser `JMP @A+DPTR` ?

Réponse FR : Utilisez `JMP @A+DPTR` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q278 — EN: When should `JMP @A+DPTR` be used?

Answer EN: Use `JMP @A+DPTR` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q279 — FR : Quel détail faut-il contrôler avant `JMP @A+DPTR` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q279 — EN: What should be checked before `JMP @A+DPTR`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q280 — FR : Comment déboguer une erreur liée à `JMP @A+DPTR` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q280 — EN: How do you debug an error involving `JMP @A+DPTR`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q281 — FR : Que fait `JZ` ?

Réponse FR : `JZ` branche si A vaut zéro.

Q281 — EN: What does `JZ` do?

Answer EN: `JZ` branches if A is zero.

Q282 — FR : Quand utiliser `JZ` ?

Réponse FR : Utilisez `JZ` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q282 — EN: When should `JZ` be used?

Answer EN: Use `JZ` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q283 — FR : Quel détail faut-il contrôler avant `JZ` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q283 — EN: What should be checked before `JZ`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q284 — FR : Comment déboguer une erreur liée à `JZ` ?**

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q284 — EN: How do you debug an error involving `JZ` ?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q285 — FR : Que fait `JNZ` ?

Réponse FR : `JNZ` branche si A n'est pas zéro.

Q285 — EN: What does `JNZ` do?

Answer EN: `JNZ` branches if A is not zero.

Q286 — FR : Quand utiliser `JNZ` ?

Réponse FR : Utilisez `JNZ` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q286 — EN: When should `JNZ` be used?

Answer EN: Use `JNZ` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q287 — FR : Quel détail faut-il contrôler avant `JNZ` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q287 — EN: What should be checked before `JNZ`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q288 — FR : Comment déboguer une erreur liée à `JNZ` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q288 — EN: How do you debug an error involving `JNZ`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q289 — FR : Que fait `JC` ?

Réponse FR : `JC` branche si CY=1.

Q289 — EN: What does `JC` do?

Answer EN: `JC` branches if CY=1.

Q290 — FR : Quand utiliser `JC` ?

Réponse FR : Utilisez `JC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q290 — EN: When should `JC` be used?

Answer EN: Use `JC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q291 — FR : Quel détail faut-il contrôler avant `JC` ?

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q291 — EN: What should be checked before `JC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q292 — FR : Comment déboguer une erreur liée à `JC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q292 — EN: How do you debug an error involving `JC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q293 — FR : Que fait `JNC` ?

Réponse FR : `JNC` branche si CY=0.

Q293 — EN: What does `JNC` do?

Answer EN: `JNC` branches if CY=0.

Q294 — FR : Quand utiliser `JNC` ?

Réponse FR : Utilisez `JNC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q294 — EN: When should `JNC` be used?

Answer EN: Use `JNC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q295 — FR : Quel détail faut-il contrôler avant `JNC` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q295 — EN: What should be checked before `JNC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q296 — FR : Comment déboguer une erreur liée à `JNC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q296 — EN: How do you debug an error involving `JNC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q297 — FR : Que fait `JB` ?

Réponse FR : `JB` branche si un bit adressable vaut 1.

Q297 — EN: What does `JB` do?

Answer EN: `JB` branches if an addressable bit is 1.

Q298 — FR : Quand utiliser `JB` ?

Réponse FR : Utilisez `JB` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q298 — EN: When should `JB` be used?**

Answer EN: Use `JB` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q299 — FR : Quel détail faut-il contrôler avant `JB` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q299 — EN: What should be checked before `JB`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q300 — FR : Comment déboguer une erreur liée à `JB` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q300 — EN: How do you debug an error involving `JB`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q301 — FR : Que fait `JNB` ?

Réponse FR : `JNB` branche si un bit adressable vaut 0.

Q301 — EN: What does `JNB` do?

Answer EN: `JNB` branches if an addressable bit is 0.

Q302 — FR : Quand utiliser `JNB` ?

Réponse FR : Utilisez `JNB` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q302 — EN: When should `JNB` be used?

Answer EN: Use `JNB` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q303 — FR : Quel détail faut-il contrôler avant `JNB` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q303 — EN: What should be checked before `JNB`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q304 — FR : Comment déboguer une erreur liée à `JNB` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q304 — EN: How do you debug an error involving `JNB`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q305 — FR : Que fait `JBC` ?

Réponse FR : `JBC` branche si bit=1 puis efface ce bit.

Q305 — EN: What does `JBC` do?

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Answer EN: `JBC` branches if bit=1 and then clears that bit.

Q306 — FR : Quand utiliser `JBC` ?

Réponse FR : Utilisez `JBC` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q306 — EN: When should `JBC` be used?

Answer EN: Use `JBC` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q307 — FR : Quel détail faut-il contrôler avant `JBC` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q307 — EN: What should be checked before `JBC`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q308 — FR : Comment déboguer une erreur liée à `JBC` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q308 — EN: How do you debug an error involving `JBC`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q309 — FR : Que fait `CJNE` ?

Réponse FR : `CJNE` compare deux opérandes supportés et branche s'ils diffèrent; CY reflète la comparaison non signée.

Q309 — EN: What does `CJNE` do?

Answer EN: `CJNE` compares supported operands and branches if unequal; CY reflects unsigned ordering.

Q310 — FR : Quand utiliser `CJNE` ?

Réponse FR : Utilisez `CJNE` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q310 — EN: When should `CJNE` be used?

Answer EN: Use `CJNE` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q311 — FR : Quel détail faut-il contrôler avant `CJNE` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q311 — EN: What should be checked before `CJNE`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q312 — FR : Comment déboguer une erreur liée à `CJNE` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q312 — EN: How do you debug an error involving `CJNE`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

listing to confirm encoding and actual destination.

Q313 — FR : Que fait `DJNZ` ?

Réponse FR : `DJNZ` décrémente un octet/Rn et branche si le résultat n'est pas zéro.

Q313 — EN: What does `DJNZ` do?

Answer EN: `DJNZ` decrements a byte/Rn and branches if result is nonzero.

Q314 — FR : Quand utiliser `DJNZ` ?

Réponse FR : Utilisez `DJNZ` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q314 — EN: When should `DJNZ` be used?

Answer EN: Use `DJNZ` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q315 — FR : Quel détail faut-il contrôler avant `DJNZ` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q315 — EN: What should be checked before `DJNZ`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q316 — FR : Comment déboguer une erreur liée à `DJNZ` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q316 — EN: How do you debug an error involving `DJNZ`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q317 — FR : Que fait `ACALL` ?

Réponse FR : `ACALL` appel court dans la page d'encodage classique, avec adresse de retour empilée.

Q317 — EN: What does `ACALL` do?

Answer EN: `ACALL` short call within classic encoding page, pushing the return address.

Q318 — FR : Quand utiliser `ACALL` ?

Réponse FR : Utilisez `ACALL` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q318 — EN: When should `ACALL` be used?

Answer EN: Use `ACALL` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q319 — FR : Quel détail faut-il contrôler avant `ACALL` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q319 — EN: What should be checked before `ACALL`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

**Q320 — FR : Comment déboguer une erreur liée à `ACALL` ?**

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q320 — EN: How do you debug an error involving `ACALL`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q321 — FR : Que fait `LCALL` ?

Réponse FR : `LCALL` appel vers une adresse CODE 16 bits avec adresse de retour empilée.

Q321 — EN: What does `LCALL` do?

Answer EN: `LCALL` call to a 16-bit CODE address with return address pushed.

Q322 — FR : Quand utiliser `LCALL` ?

Réponse FR : Utilisez `LCALL` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q322 — EN: When should `LCALL` be used?

Answer EN: Use `LCALL` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q323 — FR : Quel détail faut-il contrôler avant `LCALL` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q323 — EN: What should be checked before `LCALL`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q324 — FR : Comment déboguer une erreur liée à `LCALL` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q324 — EN: How do you debug an error involving `LCALL`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q325 — FR : Que fait `RET` ?

Réponse FR : `RET` retour de sous-routine en restaurant le PC depuis la pile.

Q325 — EN: What does `RET` do?

Answer EN: `RET` returns from a subroutine by restoring PC from the stack.

Q326 — FR : Quand utiliser `RET` ?

Réponse FR : Utilisez `RET` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q326 — EN: When should `RET` be used?

Answer EN: Use `RET` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q327 — FR : Quel détail faut-il contrôler avant `RET` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

modifie.

Q327 — EN: What should be checked before `RET`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q328 — FR : Comment déboguer une erreur liée à `RET` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q328 — EN: How do you debug an error involving `RET`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q329 — FR : Que fait `RETI` ?

Réponse FR : `RETI` retour d'interruption et signal de fin d'ISR au contrôleur d'interruptions.

Q329 — EN: What does `RETI` do?

Answer EN: `RETI` returns from interrupt and signals ISR completion to interrupt control logic.

Q330 — FR : Quand utiliser `RETI` ?

Réponse FR : Utilisez `RETI` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q330 — EN: When should `RETI` be used?

Answer EN: Use `RETI` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q331 — FR : Quel détail faut-il contrôler avant `RETI` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q331 — EN: What should be checked before `RETI`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q332 — FR : Comment déboguer une erreur liée à `RETI` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q332 — EN: How do you debug an error involving `RETI`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

Q333 — FR : Que fait `NOP` ?

Réponse FR : `NOP` n'effectue aucune opération fonctionnelle autre que consommer son temps d'instruction.

Q333 — EN: What does `NOP` do?

Answer EN: `NOP` performs no functional operation other than consuming its instruction time.

Q334 — FR : Quand utiliser `NOP` ?

Réponse FR : Utilisez `NOP` lorsque son mode d'adressage et ses effets sur A/CY/flags correspondent exactement à l'algorithme. Vérifiez les variantes d'opérandes autorisées dans la table d'instructions.

Q334 — EN: When should `NOP` be used?



Answer EN: Use `NOP` when its addressing mode and effects on A/CY/flags exactly match the algorithm. Verify legal operand variants in the instruction table.

Q335 — FR : Quel détail faut-il contrôler avant `NOP` ?

Réponse FR : Contrôler la bank active, l'espace mémoire visé, la valeur de SP/DPTR si pertinente, et les flags que l'instruction lit ou modifie.

Q335 — EN: What should be checked before `NOP`?

Answer EN: Check the active register bank, target memory space, SP/DPTR where relevant, and any flags the instruction reads or modifies.

Q336 — FR : Comment déboguer une erreur liée à `NOP` ?

Réponse FR : Single-step sur l'instruction et comparer avant/après A, B, CY, PSW, DPTR, SP et l'octet mémoire concerné. Utiliser le listing pour confirmer l'encodage et la destination réelle.

Q336 — EN: How do you debug an error involving `NOP`?

Answer EN: Single-step the instruction and compare A, B, CY, PSW, DPTR, SP, and the affected memory byte before/after. Use the listing to confirm encoding and actual destination.

6. Practical Assembly patterns / Patterns Assembly pratiques

Q337 — FR : Comment sélectionner la bank 0 en Assembly 8051 ?

Réponse FR : Principe : clear RS1 and RS0 so R0-R7 map to 00h-07h. Exemple : `CLR PSW.4 / CLR PSW.3`. Adaptez les adresses à la memory map réelle.

Q337 — EN: How do you sélectionner la bank 0 in 8051 Assembly?

Answer EN: Principle: clear RS1 and RS0 so R0-R7 map to 00h-07h. Example: `CLR PSW.4 / CLR PSW.3`. Adapt addresses to the actual device memory map.

Q338 — FR : Quelle erreur classique peut apparaître lorsqu'on veut sélectionner la bank 0 ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q338 — EN: What classic error can occur when trying to sélectionner la bank 0?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q339 — FR : Comment valider le code qui sert à sélectionner la bank 0 ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q339 — EN: How do you validate code used to sélectionner la bank 0?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q340 — FR : Comment sélectionner la bank 1 en Assembly 8051 ?

Réponse FR : Principe : set RS0 and clear RS1 so R0-R7 map to 08h-0Fh. Exemple : `CLR PSW.4 / SETB PSW.3`. Adaptez les adresses à la memory map réelle.

Q340 — EN: How do you sélectionner la bank 1 in 8051 Assembly?

Answer EN: Principle: set RS0 and clear RS1 so R0-R7 map to 08h-0Fh. Example: `CLR PSW.4 / SETB PSW.3`. Adapt addresses to the actual device memory map.

**Q341 — FR : Quelle erreur classique peut apparaître lorsqu'on veut sélectionner la bank 1 ?**

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q341 — EN: What classic error can occur when trying to sélectionner la bank 1?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q342 — FR : Comment valider le code qui sert à sélectionner la bank 1 ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q342 — EN: How do you validate code used to sélectionner la bank 1?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q343 — FR : Comment initialiser la pile en Assembly 8051 ?

Réponse FR : Principe : move the stack above selected internal-RAM variables/register banks when the device RAM map permits. Exemple : `MOV SP,#2Fh`. Adaptez les adresses à la memory map réelle.

Q343 — EN: How do you initialiser la pile in 8051 Assembly?

Answer EN: Principle: move the stack above selected internal-RAM variables/register banks when the device RAM map permits. Example: `MOV SP,#2Fh`. Adapt addresses to the actual device memory map.

Q344 — FR : Quelle erreur classique peut apparaître lorsqu'on veut initialiser la pile ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q344 — EN: What classic error can occur when trying to initialiser la pile?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q345 — FR : Comment valider le code qui sert à initialiser la pile ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q345 — EN: How do you validate code used to initialiser la pile?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q346 — FR : Comment copier une constante dans A en Assembly 8051 ?

Réponse FR : Principe : load an immediate constant into the accumulator. Exemple : `MOV A,#55h`. Adaptez les adresses à la memory map réelle.

Q346 — EN: How do you copier une constante dans A in 8051 Assembly?

Answer EN: Principle: load an immediate constant into the accumulator. Example: `MOV A,#55h`. Adapt addresses to the actual device memory map.

Q347 — FR : Quelle erreur classique peut apparaître lorsqu'on veut copier une constante dans A ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q347 — EN: What classic error can occur when trying to copier une constante dans A?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller



expected preserved.

Q348 — FR : Comment valider le code qui sert à copier une constante dans A ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q348 — EN: How do you validate code used to copier une constante dans A?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q349 — FR : Comment écrire une variable DATA en Assembly 8051 ?

Réponse FR : Principe : write a direct internal-RAM byte. Exemple : ``MOV 30h,#12h``. Adaptez les adresses à la memory map réelle.

Q349 — EN: How do you écrire une variable DATA in 8051 Assembly?

Answer EN: Principle: write a direct internal-RAM byte. Example: ``MOV 30h,#12h``. Adapt addresses to the actual device memory map.

Q350 — FR : Quelle erreur classique peut apparaître lorsqu'on veut écrire une variable DATA ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q350 — EN: What classic error can occur when trying to écrire une variable DATA?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q351 — FR : Comment valider le code qui sert à écrire une variable DATA ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q351 — EN: How do you validate code used to écrire une variable DATA?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q352 — FR : Comment lire indirectement la RAM en Assembly 8051 ?

Réponse FR : Principe : put an internal-RAM address in R0 and read through it. Exemple : ``MOV R0,#30h / MOV A,@R0``. Adaptez les adresses à la memory map réelle.

Q352 — EN: How do you lire indirectement la RAM in 8051 Assembly?

Answer EN: Principle: put an internal-RAM address in R0 and read through it. Example: ``MOV R0,#30h / MOV A,@R0``. Adapt addresses to the actual device memory map.

Q353 — FR : Quelle erreur classique peut apparaître lorsqu'on veut lire indirectement la RAM ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q353 — EN: What classic error can occur when trying to lire indirectement la RAM?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q354 — FR : Comment valider le code qui sert à lire indirectement la RAM ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q354 — EN: How do you validate code used to lire indirectement la RAM?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a



simple case and then boundary values.

Q355 — FR : Comment charger DPTR en Assembly 8051 ?

Réponse FR : Principe : load a 16-bit address into DPTR. Exemple : ``MOV DPTR,#1234h``. Adaptez les adresses à la memory map réelle.

Q355 — EN: How do you charger DPTR in 8051 Assembly?

Answer EN: Principle: load a 16-bit address into DPTR. Example: ``MOV DPTR,#1234h``. Adapt addresses to the actual device memory map.

Q356 — FR : Quelle erreur classique peut apparaître lorsqu'on veut charger DPTR ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q356 — EN: What classic error can occur when trying to charger DPTR?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q357 — FR : Comment valider le code qui sert à charger DPTR ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q357 — EN: How do you validate code used to charger DPTR?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q358 — FR : Comment lire une table CODE en Assembly 8051 ?

Réponse FR : Principe : use A as the byte index into a CODE table. Exemple : ``MOV DPTR,#TABLE / MOV A,R0 / MOVC A,@A+DPTR``. Adaptez les adresses à la memory map réelle.

Q358 — EN: How do you lire une table CODE in 8051 Assembly?

Answer EN: Principle: use A as the byte index into a CODE table. Example: ``MOV DPTR,#TABLE / MOV A,R0 / MOVC A,@A+DPTR``. Adapt addresses to the actual device memory map.

Q359 — FR : Quelle erreur classique peut apparaître lorsqu'on veut lire une table CODE ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q359 — EN: What classic error can occur when trying to lire une table CODE?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q360 — FR : Comment valider le code qui sert à lire une table CODE ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q360 — EN: How do you validate code used to lire une table CODE?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q361 — FR : Comment additionner 16 bits en Assembly 8051 ?

Réponse FR : Principe : add low bytes with ADD then high bytes with ADDC to propagate CY. Exemple : ``MOV A,R4 / ADD A,R6 / MOV R0,A / MOV A,R5 / ADDC A,R7``. Adaptez les adresses à la memory map réelle.

Q361 — EN: How do you additionner 16 bits in 8051 Assembly?

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Answer EN: Principle: add low bytes with ADD then high bytes with ADDC to propagate CY. Example: `MOV A,R4 / ADD A,R6 / MOV R0,A / MOV A,R5 / ADDC A,R7`. Adapt addresses to the actual device memory map.

Q362 — FR : Quelle erreur classique peut apparaître lorsqu'on veut additionner 16 bits ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q362 — EN: What classic error can occur when trying to additionner 16 bits?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q363 — FR : Comment valider le code qui sert à additionner 16 bits ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q363 — EN: How do you validate code used to additionner 16 bits?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q364 — FR : Comment soustraire 16 bits en Assembly 8051 ?

Réponse FR : Principe : clear borrow first, subtract low byte, then propagate CY/borrow into high byte. Exemple : `CLR C / MOV A,R4 / SUBB A,R6 / MOV R0,A / MOV A,R5 / SUBB A,R7`. Adaptez les adresses à la memory map réelle.

Q364 — EN: How do you soustraire 16 bits in 8051 Assembly?

Answer EN: Principle: clear borrow first, subtract low byte, then propagate CY/borrow into high byte. Example: `CLR C / MOV A,R4 / SUBB A,R6 / MOV R0,A / MOV A,R5 / SUBB A,R7`. Adapt addresses to the actual device memory map.

Q365 — FR : Quelle erreur classique peut apparaître lorsqu'on veut soustraire 16 bits ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q365 — EN: What classic error can occur when trying to soustraire 16 bits?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q366 — FR : Comment valider le code qui sert à soustraire 16 bits ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q366 — EN: How do you validate code used to soustraire 16 bits?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q367 — FR : Comment tester un bit en Assembly 8051 ?

Réponse FR : Principe : branch directly on a bit-addressable flag. Exemple : `JB 20h.0,IS\_SET`. Adaptez les adresses à la memory map réelle.

Q367 — EN: How do you tester un bit in 8051 Assembly?

Answer EN: Principle: branch directly on a bit-addressable flag. Example: `JB 20h.0,IS\_SET`. Adapt addresses to the actual device memory map.

Q368 — FR : Quelle erreur classique peut apparaître lorsqu'on veut tester un bit ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que



l'appelant attendait préservée.

Q368 — EN: What classic error can occur when trying to tester un bit?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q369 — FR : Comment valider le code qui sert à tester un bit ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q369 — EN: How do you validate code used to tester un bit?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q370 — FR : Comment boucle comptée en Assembly 8051 ?

Réponse FR : Principe : use DJNZ as a compact countdown loop. Exemple : `MOV R7,#100 / LOOP1: NOP / DJNZ R7,LOOP1`. Adaptez les adresses à la memory map réelle.

Q370 — EN: How do you boucle comptée in 8051 Assembly?

Answer EN: Principle: use DJNZ as a compact countdown loop. Example: `MOV R7,#100 / LOOP1: NOP / DJNZ R7,LOOP1`. Adapt addresses to the actual device memory map.

Q371 — FR : Quelle erreur classique peut apparaître lorsqu'on veut boucle comptée ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q371 — EN: What classic error can occur when trying to boucle comptée?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q372 — FR : Comment valider le code qui sert à boucle comptée ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q372 — EN: How do you validate code used to boucle comptée?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q373 — FR : Comment appel de sous-routine en Assembly 8051 ?

Réponse FR : Principe : push the return address automatically and return with RET. Exemple : `LCALL PROCESS / ... / PROCESS: / ; body / RET`. Adaptez les adresses à la memory map réelle.

Q373 — EN: How do you appel de sous-routine in 8051 Assembly?

Answer EN: Principle: push the return address automatically and return with RET. Example: `LCALL PROCESS / ... / PROCESS: / ; body / RET`. Adapt addresses to the actual device memory map.

Q374 — FR : Quelle erreur classique peut apparaître lorsqu'on veut appel de sous-routine ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q374 — EN: What classic error can occur when trying to appel de sous-routine?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q375 — FR : Comment valider le code qui sert à appel de sous-routine ?



Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q375 — EN: How do you validate code used to appel de sous-routine?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q376 — FR : Comment sauver A et PSW en Assembly 8051 ?

Réponse FR : Principe : preserve registers before code that must restore caller state. Exemple : `PUSH ACC / PUSH PSW`. Adaptez les adresses à la memory map réelle.

Q376 — EN: How do you sauver A et PSW in 8051 Assembly?

Answer EN: Principle: preserve registers before code that must restore caller state. Example: `PUSH ACC / PUSH PSW`. Adapt addresses to the actual device memory map.

Q377 — FR : Quelle erreur classique peut apparaître lorsqu'on veut sauver A et PSW ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q377 — EN: What classic error can occur when trying to sauver A et PSW?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q378 — FR : Comment valider le code qui sert à sauver A et PSW ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q378 — EN: How do you validate code used to sauver A et PSW?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q379 — FR : Comment restaurer PSW et A en Assembly 8051 ?

Réponse FR : Principe : restore in reverse order because the stack is LIFO. Exemple : `POP PSW / POP ACC`. Adaptez les adresses à la memory map réelle.

Q379 — EN: How do you restaurer PSW et A in 8051 Assembly?

Answer EN: Principle: restore in reverse order because the stack is LIFO. Example: `POP PSW / POP ACC`. Adapt addresses to the actual device memory map.

Q380 — FR : Quelle erreur classique peut apparaître lorsqu'on veut restaurer PSW et A ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q380 — EN: What classic error can occur when trying to restaurer PSW et A?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q381 — FR : Comment valider le code qui sert à restaurer PSW et A ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q381 — EN: How do you validate code used to restaurer PSW et A?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a



simple case and then boundary values.

Q382 — FR : Comment table de dispatch en Assembly 8051 ?

Réponse FR : Principe : index a compact jump table when entries are arranged for the chosen encoding. Exemple : `MOV DPTR,#JTABLE / MOV A,R0 / RL A / JMP @A+DPTR`. Adaptez les adresses à la memory map réelle.

Q382 — EN: How do you table de dispatch in 8051 Assembly?

Answer EN: Principle: index a compact jump table when entries are arranged for the chosen encoding. Example: `MOV DPTR,#JTABLE / MOV A,R0 / RL A / JMP @A+DPTR`. Adapt addresses to the actual device memory map.

Q383 — FR : Quelle erreur classique peut apparaître lorsqu'on veut table de dispatch ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q383 — EN: What classic error can occur when trying to table de dispatch?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q384 — FR : Comment valider le code qui sert à table de dispatch ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q384 — EN: How do you validate code used to table de dispatch?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q385 — FR : Comment masquer des bits en Assembly 8051 ?

Réponse FR : Principe : keep only selected bits using a mask. Exemple : `ANL A,#0Fh`. Adaptez les adresses à la memory map réelle.

Q385 — EN: How do you masquer des bits in 8051 Assembly?

Answer EN: Principle: keep only selected bits using a mask. Example: `ANL A,#0Fh`. Adapt addresses to the actual device memory map.

Q386 — FR : Quelle erreur classique peut apparaître lorsqu'on veut masquer des bits ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q386 — EN: What classic error can occur when trying to masquer des bits?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q387 — FR : Comment valider le code qui sert à masquer des bits ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q387 — EN: How do you validate code used to masquer des bits?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q388 — FR : Comment mettre des bits en Assembly 8051 ?

Réponse FR : Principe : set selected bits while preserving others. Exemple : `ORL A,#80h`. Adaptez les adresses à la memory map réelle.

Q388 — EN: How do you mettre des bits in 8051 Assembly?

Answer EN: Principle: set selected bits while preserving others. Example: `ORL A,#80h`. Adapt addresses to the actual device memory

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

map.

Q389 — FR : Quelle erreur classique peut apparaître lorsqu'on veut mettre des bits ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q389 — EN: What classic error can occur when trying to mettre des bits?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q390 — FR : Comment valider le code qui sert à mettre des bits ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q390 — EN: How do you validate code used to mettre des bits?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q391 — FR : Comment basculer des bits en Assembly 8051 ?

Réponse FR : Principe : toggle selected bits with XOR. Exemple : `XRL A,#01h`. Adaptez les adresses à la memory map réelle.

Q391 — EN: How do you basculer des bits in 8051 Assembly?

Answer EN: Principle: toggle selected bits with XOR. Example: `XRL A,#01h`. Adapt addresses to the actual device memory map.

Q392 — FR : Quelle erreur classique peut apparaître lorsqu'on veut basculer des bits ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q392 — EN: What classic error can occur when trying to basculer des bits?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller expected preserved.

Q393 — FR : Comment valider le code qui sert à basculer des bits ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q393 — EN: How do you validate code used to basculer des bits?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

Q394 — FR : Comment rotation 16 bits en Assembly 8051 ?

Réponse FR : Principe : propagate CY through multiple bytes for a multi-byte shift/rotate. Exemple : `MOV A,R0 / RLC A / MOV R0,A / MOV A,R1 / RLC A / MOV R1,A`. Adaptez les adresses à la memory map réelle.

Q394 — EN: How do you rotation 16 bits in 8051 Assembly?

Answer EN: Principle: propagate CY through multiple bytes for a multi-byte shift/rotate. Example: `MOV A,R0 / RLC A / MOV R0,A / MOV A,R1 / RLC A / MOV R1,A`. Adapt addresses to the actual device memory map.

Q395 — FR : Quelle erreur classique peut apparaître lorsqu'on veut rotation 16 bits ?

Réponse FR : Utiliser une adresse appartenant au mauvais espace, oublier un flag/registre implicite, ou écraser une valeur que l'appelant attendait préservée.

Q395 — EN: What classic error can occur when trying to rotation 16 bits?

Answer EN: Using an address from the wrong memory space, forgetting an implicit flag/register, or overwriting a value the caller

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

expected preserved.

Q396 — FR : Comment valider le code qui sert à rotation 16 bits ?

Réponse FR : Assembler avec listing, vérifier les adresses symboliques, exécuter pas à pas et comparer les registres/mémoires avant et après avec un cas simple puis des valeurs limites.

Q396 — EN: How do you validate code used to rotation 16 bits?

Answer EN: Assemble with a listing, verify symbolic addresses, single-step, and compare registers/memory before and after using a simple case and then boundary values.

7. Timing, stack, robustness and interview scenarios / Timing, pile, robustesse et entretien

Q397 — FR : Pourquoi la pile 8051 demande-t-elle une planification explicite ?

Réponse FR : SP est petit et la pile partage typiquement la RAM interne avec variables et banks. Les CALL, interrupts et PUSH peuvent provoquer une collision si SP et la consommation maximale ne sont pas calculés.

Q397 — EN: Why does the 8051 stack require explicit planning?

Answer EN: SP is small and the stack typically shares internal RAM with variables and register banks. CALLs, interrupts, and PUSH operations can collide unless SP and worst-case depth are planned.

Q398 — FR : Entretien — Pourquoi la pile 8051 demande-t-elle une planification explicite ?

Réponse FR : SP est petit et la pile partage typiquement la RAM interne avec variables et banks. Les CALL, interrupts et PUSH peuvent provoquer une collision si SP et la consommation maximale ne sont pas calculés.

Q398 — EN: Entretien — Why does the 8051 stack require explicit planning?

Answer EN: SP is small and the stack typically shares internal RAM with variables and register banks. CALLs, interrupts, and PUSH operations can collide unless SP and worst-case depth are planned.

Q399 — FR : Debug — Pourquoi la pile 8051 demande-t-elle une planification explicite ?

Réponse FR : SP est petit et la pile partage typiquement la RAM interne avec variables et banks. Les CALL, interrupts et PUSH peuvent provoquer une collision si SP et la consommation maximale ne sont pas calculés.

Q399 — EN: Debug — Why does the 8051 stack require explicit planning?

Answer EN: SP is small and the stack typically shares internal RAM with variables and register banks. CALLs, interrupts, and PUSH operations can collide unless SP and worst-case depth are planned.

Q400 — FR : Code review — Pourquoi la pile 8051 demande-t-elle une planification explicite ?

Réponse FR : SP est petit et la pile partage typiquement la RAM interne avec variables et banks. Les CALL, interrupts et PUSH peuvent provoquer une collision si SP et la consommation maximale ne sont pas calculés.

Q400 — EN: Code review — Why does the 8051 stack require explicit planning?

Answer EN: SP is small and the stack typically shares internal RAM with variables and register banks. CALLs, interrupts, and PUSH operations can collide unless SP and worst-case depth are planned.

Q401 — FR : Pourquoi faut-il distinguer machine cycle et oscillator clock ?

Réponse FR : Le 8051 original utilisait classiquement plusieurs clocks par machine cycle, alors que des dérivés accélérés comme certains NXP LPC9xx exécutent plus rapidement. Les délais doivent être calculés avec le datasheet du composant réel.

Q401 — EN: Why distinguish machine cycle from oscillator clock?

Answer EN: The original 8051 classically used multiple clocks per machine cycle, while accelerated derivatives such as some NXP LPC9xx execute faster. Delay calculations must use the actual device data sheet.

**Q402 — FR : Entretien — Pourquoi faut-il distinguer machine cycle et oscillator clock ?**

Réponse FR : Le 8051 original utilisait classiquement plusieurs clocks par machine cycle, alors que des dérivés accélérés comme certains NXP LPC9xx exécutent plus rapidement. Les délais doivent être calculés avec le datasheet du composant réel.

Q402 — EN: Entretien — Why distinguish machine cycle from oscillator clock?

Answer EN: The original 8051 classically used multiple clocks per machine cycle, while accelerated derivatives such as some NXP LPC9xx execute faster. Delay calculations must use the actual device data sheet.

Q403 — FR : Debug — Pourquoi faut-il distinguer machine cycle et oscillator clock ?

Réponse FR : Le 8051 original utilisait classiquement plusieurs clocks par machine cycle, alors que des dérivés accélérés comme certains NXP LPC9xx exécutent plus rapidement. Les délais doivent être calculés avec le datasheet du composant réel.

Q403 — EN: Debug — Why distinguish machine cycle from oscillator clock?

Answer EN: The original 8051 classically used multiple clocks per machine cycle, while accelerated derivatives such as some NXP LPC9xx execute faster. Delay calculations must use the actual device data sheet.

Q404 — FR : Code review — Pourquoi faut-il distinguer machine cycle et oscillator clock ?

Réponse FR : Le 8051 original utilisait classiquement plusieurs clocks par machine cycle, alors que des dérivés accélérés comme certains NXP LPC9xx exécutent plus rapidement. Les délais doivent être calculés avec le datasheet du composant réel.

Q404 — EN: Code review — Why distinguish machine cycle from oscillator clock?

Answer EN: The original 8051 classically used multiple clocks per machine cycle, while accelerated derivatives such as some NXP LPC9xx execute faster. Delay calculations must use the actual device data sheet.

Q405 — FR : Pourquoi une boucle de délai Assembly est-elle fragile ?

Réponse FR : Sa durée dépend du core, de la fréquence et du timing exact de chaque instruction. Une migration vers un dérivé two-clock peut changer fortement le délai.

Q405 — EN: Why is an Assembly delay loop fragile?

Answer EN: Its duration depends on the core, frequency, and exact instruction timing. Moving to a two-clock derivative can change the delay substantially.

Q406 — FR : Entretien — Pourquoi une boucle de délai Assembly est-elle fragile ?

Réponse FR : Sa durée dépend du core, de la fréquence et du timing exact de chaque instruction. Une migration vers un dérivé two-clock peut changer fortement le délai.

Q406 — EN: Entretien — Why is an Assembly delay loop fragile?

Answer EN: Its duration depends on the core, frequency, and exact instruction timing. Moving to a two-clock derivative can change the delay substantially.

Q407 — FR : Debug — Pourquoi une boucle de délai Assembly est-elle fragile ?

Réponse FR : Sa durée dépend du core, de la fréquence et du timing exact de chaque instruction. Une migration vers un dérivé two-clock peut changer fortement le délai.

Q407 — EN: Debug — Why is an Assembly delay loop fragile?

Answer EN: Its duration depends on the core, frequency, and exact instruction timing. Moving to a two-clock derivative can change the delay substantially.

Q408 — FR : Code review — Pourquoi une boucle de délai Assembly est-elle fragile ?

Réponse FR : Sa durée dépend du core, de la fréquence et du timing exact de chaque instruction. Une migration vers un dérivé two-clock peut changer fortement le délai.

Q408 — EN: Code review — Why is an Assembly delay loop fragile?

Answer EN: Its duration depends on the core, frequency, and exact instruction timing. Moving to a two-clock derivative can change

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

the delay substantially.

Q409 — FR : Comment faire une arithmétique 24 ou 32 bits sur un CPU 8 bits ?

Réponse FR : Traiter les octets du poids faible vers le poids fort avec ADD/ADDC ou SUBB et propager CY. Définir clairement endianness et emplacement des octets.

Q409 — EN: How do you perform 24- or 32-bit arithmetic on an 8-bit CPU?

Answer EN: Process bytes from least significant to most significant using ADD/ADDC or SUBB and propagate CY. Define byte order and storage explicitly.

Q410 — FR : Entretien — Comment faire une arithmétique 24 ou 32 bits sur un CPU 8 bits ?

Réponse FR : Traiter les octets du poids faible vers le poids fort avec ADD/ADDC ou SUBB et propager CY. Définir clairement endianness et emplacement des octets.

Q410 — EN: Entretien — How do you perform 24- or 32-bit arithmetic on an 8-bit CPU?

Answer EN: Process bytes from least significant to most significant using ADD/ADDC or SUBB and propagate CY. Define byte order and storage explicitly.

Q411 — FR : Debug — Comment faire une arithmétique 24 ou 32 bits sur un CPU 8 bits ?

Réponse FR : Traiter les octets du poids faible vers le poids fort avec ADD/ADDC ou SUBB et propager CY. Définir clairement endianness et emplacement des octets.

Q411 — EN: Debug — How do you perform 24- or 32-bit arithmetic on an 8-bit CPU?

Answer EN: Process bytes from least significant to most significant using ADD/ADDC or SUBB and propagate CY. Define byte order and storage explicitly.

Q412 — FR : Code review — Comment faire une arithmétique 24 ou 32 bits sur un CPU 8 bits ?

Réponse FR : Traiter les octets du poids faible vers le poids fort avec ADD/ADDC ou SUBB et propager CY. Définir clairement endianness et emplacement des octets.

Q412 — EN: Code review — How do you perform 24- or 32-bit arithmetic on an 8-bit CPU?

Answer EN: Process bytes from least significant to most significant using ADD/ADDC or SUBB and propagate CY. Define byte order and storage explicitly.

Q413 — FR : Pourquoi CJNE est-il utile au-delà du simple != ?

Réponse FR : Après comparaison, CY peut indiquer l'ordre non signé, ce qui permet de construire des tests inférieur/supérieur avec branchement.

Q413 — EN: Why is CJNE useful beyond simple !=?

Answer EN: After comparison, CY can indicate unsigned ordering, allowing less-than/greater-than logic to be constructed.

Q414 — FR : Entretien — Pourquoi CJNE est-il utile au-delà du simple != ?

Réponse FR : Après comparaison, CY peut indiquer l'ordre non signé, ce qui permet de construire des tests inférieur/supérieur avec branchement.

Q414 — EN: Entretien — Why is CJNE useful beyond simple !=?

Answer EN: After comparison, CY can indicate unsigned ordering, allowing less-than/greater-than logic to be constructed.

Q415 — FR : Debug — Pourquoi CJNE est-il utile au-delà du simple != ?

Réponse FR : Après comparaison, CY peut indiquer l'ordre non signé, ce qui permet de construire des tests inférieur/supérieur avec branchement.

Q415 — EN: Debug — Why is CJNE useful beyond simple !=?

Answer EN: After comparison, CY can indicate unsigned ordering, allowing less-than/greater-than logic to be constructed.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q416 — FR : Code review — Pourquoi CJNE est-il utile au-delà du simple != ?**

Réponse FR : Après comparaison, CY peut indiquer l'ordre non signé, ce qui permet de construire des tests inférieur/supérieur avec branchement.

Q416 — EN: Code review — Why is CJNE useful beyond simple !=?

Answer EN: After comparison, CY can indicate unsigned ordering, allowing less-than/greater-than logic to be constructed.

Q417 — FR : Pourquoi JBC est-il utile pour des flags logiciels ?

Réponse FR : Il teste un bit, branche s'il était à 1 et le remet à 0 de façon compacte, utile pour consommer un événement représenté par un flag bit-adressable.

Q417 — EN: Why is JBC useful for software flags?

Answer EN: It tests a bit, branches if it was 1, and clears it compactly, useful for consuming an event represented by a bit-addressable flag.

Q418 — FR : Entretien — Pourquoi JBC est-il utile pour des flags logiciels ?

Réponse FR : Il teste un bit, branche s'il était à 1 et le remet à 0 de façon compacte, utile pour consommer un événement représenté par un flag bit-adressable.

Q418 — EN: Entretien — Why is JBC useful for software flags?

Answer EN: It tests a bit, branches if it was 1, and clears it compactly, useful for consuming an event represented by a bit-addressable flag.

Q419 — FR : Debug — Pourquoi JBC est-il utile pour des flags logiciels ?

Réponse FR : Il teste un bit, branche s'il était à 1 et le remet à 0 de façon compacte, utile pour consommer un événement représenté par un flag bit-adressable.

Q419 — EN: Debug — Why is JBC useful for software flags?

Answer EN: It tests a bit, branches if it was 1, and clears it compactly, useful for consuming an event represented by a bit-addressable flag.

Q420 — FR : Code review — Pourquoi JBC est-il utile pour des flags logiciels ?

Réponse FR : Il teste un bit, branche s'il était à 1 et le remet à 0 de façon compacte, utile pour consommer un événement représenté par un flag bit-adressable.

Q420 — EN: Code review — Why is JBC useful for software flags?

Answer EN: It tests a bit, branches if it was 1, and clears it compactly, useful for consuming an event represented by a bit-addressable flag.

Q421 — FR : RET et RETI sont-ils interchangeables ?

Réponse FR : Non. RET restaure seulement le flux d'un appel normal; RETI est destiné à la sortie d'une ISR et informe aussi la logique d'interruption selon l'architecture.

Q421 — EN: Are RET and RETI interchangeable?

Answer EN: No. RET returns from a normal call; RETI is intended for ISR exit and also informs interrupt-control logic according to the architecture.

Q422 — FR : Entretien — RET et RETI sont-ils interchangeables ?

Réponse FR : Non. RET restaure seulement le flux d'un appel normal; RETI est destiné à la sortie d'une ISR et informe aussi la logique d'interruption selon l'architecture.

Q422 — EN: Entretien — Are RET and RETI interchangeable?

Answer EN: No. RET returns from a normal call; RETI is intended for ISR exit and also informs interrupt-control logic according to the

**AHDS**Annisoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

architecture.

Q423 — FR : Debug — RET et RETI sont-ils interchangeables ?

Réponse FR : Non. RET restaure seulement le flux d'un appel normal; RETI est destiné à la sortie d'une ISR et informe aussi la logique d'interruption selon l'architecture.

Q423 — EN: Debug — Are RET and RETI interchangeable?

Answer EN: No. RET returns from a normal call; RETI is intended for ISR exit and also informs interrupt-control logic according to the architecture.

Q424 — FR : Code review — RET et RETI sont-ils interchangeables ?

Réponse FR : Non. RET restaure seulement le flux d'un appel normal; RETI est destiné à la sortie d'une ISR et informe aussi la logique d'interruption selon l'architecture.

Q424 — EN: Code review — Are RET and RETI interchangeable?

Answer EN: No. RET returns from a normal call; RETI is intended for ISR exit and also informs interrupt-control logic according to the architecture.

Q425 — FR : Pourquoi MOV direct,direct est-il intéressant sur 8051 ?

Réponse FR : Certaines formes MOV permettent de copier directement entre emplacements sans utiliser A, ce qui peut préserver l'accumulateur et réduire le code de sauvegarde.

Q425 — EN: Why is MOV direct,direct useful on the 8051?

Answer EN: Some MOV forms copy directly between locations without using A, which can preserve the accumulator and reduce save/restore code.

Q426 — FR : Entretien — Pourquoi MOV direct,direct est-il intéressant sur 8051 ?

Réponse FR : Certaines formes MOV permettent de copier directement entre emplacements sans utiliser A, ce qui peut préserver l'accumulateur et réduire le code de sauvegarde.

Q426 — EN: Entretien — Why is MOV direct,direct useful on the 8051?

Answer EN: Some MOV forms copy directly between locations without using A, which can preserve the accumulator and reduce save/restore code.

Q427 — FR : Debug — Pourquoi MOV direct,direct est-il intéressant sur 8051 ?

Réponse FR : Certaines formes MOV permettent de copier directement entre emplacements sans utiliser A, ce qui peut préserver l'accumulateur et réduire le code de sauvegarde.

Q427 — EN: Debug — Why is MOV direct,direct useful on the 8051?

Answer EN: Some MOV forms copy directly between locations without using A, which can preserve the accumulator and reduce save/restore code.

Q428 — FR : Code review — Pourquoi MOV direct,direct est-il intéressant sur 8051 ?

Réponse FR : Certaines formes MOV permettent de copier directement entre emplacements sans utiliser A, ce qui peut préserver l'accumulateur et réduire le code de sauvegarde.

Q428 — EN: Code review — Why is MOV direct,direct useful on the 8051?

Answer EN: Some MOV forms copy directly between locations without using A, which can preserve the accumulator and reduce save/restore code.

Q429 — FR : Comment lire une constante stockée dans le programme ?

Réponse FR : Utiliser MOVC, pas MOV/MOVB. Charger DPTR sur la table, mettre l'index dans A puis MOVC A,@A+DPTR.

Q429 — EN: How do you read a constant stored in program memory?



Answer EN: Use MOVC, not MOV/MOVX. Point DPTR at the table, place the index in A, then execute MOVC A,@A+DPTR.

Q430 — FR : Entretien — Comment lire une constante stockée dans le programme ?

Réponse FR : Utiliser MOVC, pas MOV/MOVX. Charger DPTR sur la table, mettre l'index dans A puis MOVC A,@A+DPTR.

Q430 — EN: Entretien — How do you read a constant stored in program memory?

Answer EN: Use MOVC, not MOV/MOVX. Point DPTR at the table, place the index in A, then execute MOVC A,@A+DPTR.

Q431 — FR : Debug — Comment lire une constante stockée dans le programme ?

Réponse FR : Utiliser MOVC, pas MOV/MOVX. Charger DPTR sur la table, mettre l'index dans A puis MOVC A,@A+DPTR.

Q431 — EN: Debug — How do you read a constant stored in program memory?

Answer EN: Use MOVC, not MOV/MOVX. Point DPTR at the table, place the index in A, then execute MOVC A,@A+DPTR.

Q432 — FR : Code review — Comment lire une constante stockée dans le programme ?

Réponse FR : Utiliser MOVC, pas MOV/MOVX. Charger DPTR sur la table, mettre l'index dans A puis MOVC A,@A+DPTR.

Q432 — EN: Code review — How do you read a constant stored in program memory?

Answer EN: Use MOVC, not MOV/MOVX. Point DPTR at the table, place the index in A, then execute MOVC A,@A+DPTR.

Q433 — FR : Pourquoi les symboles SFR doivent-ils venir du bon fichier device ?

Réponse FR : Les SFR du cœur classique sont relativement stables, mais les périphériques et adresses supplémentaires varient selon le dérivé. Un mauvais include peut assembler un code valide mais viser le mauvais registre.

Q433 — EN: Why must SFR symbols come from the correct device file?

Answer EN: Classic-core SFRs are relatively stable, but peripheral SFRs and extensions vary by derivative. The wrong include can assemble valid code that targets the wrong register.

Q434 — FR : Entretien — Pourquoi les symboles SFR doivent-ils venir du bon fichier device ?

Réponse FR : Les SFR du cœur classique sont relativement stables, mais les périphériques et adresses supplémentaires varient selon le dérivé. Un mauvais include peut assembler un code valide mais viser le mauvais registre.

Q434 — EN: Entretien — Why must SFR symbols come from the correct device file?

Answer EN: Classic-core SFRs are relatively stable, but peripheral SFRs and extensions vary by derivative. The wrong include can assemble valid code that targets the wrong register.

Q435 — FR : Debug — Pourquoi les symboles SFR doivent-ils venir du bon fichier device ?

Réponse FR : Les SFR du cœur classique sont relativement stables, mais les périphériques et adresses supplémentaires varient selon le dérivé. Un mauvais include peut assembler un code valide mais viser le mauvais registre.

Q435 — EN: Debug — Why must SFR symbols come from the correct device file?

Answer EN: Classic-core SFRs are relatively stable, but peripheral SFRs and extensions vary by derivative. The wrong include can assemble valid code that targets the wrong register.

Q436 — FR : Code review — Pourquoi les symboles SFR doivent-ils venir du bon fichier device ?

Réponse FR : Les SFR du cœur classique sont relativement stables, mais les périphériques et adresses supplémentaires varient selon le dérivé. Un mauvais include peut assembler un code valide mais viser le mauvais registre.

Q436 — EN: Code review — Why must SFR symbols come from the correct device file?

Answer EN: Classic-core SFRs are relatively stable, but peripheral SFRs and extensions vary by derivative. The wrong include can assemble valid code that targets the wrong register.

Q437 — FR : Comment documenter une routine Assembly proprement ?

Réponse FR : Indiquer entrées, sorties, registres détruits/préservés, flags modifiés, mémoire utilisée, profondeur de pile, timing

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

critique et hypothèses hardware.

Q437 — EN: How should an Assembly routine be documented?

Answer EN: State inputs, outputs, clobbered/preserved registers, modified flags, memory used, stack depth, critical timing, and hardware assumptions.

Q438 — FR : Entretien — Comment documenter une routine Assembly proprement ?

Réponse FR : Indiquer entrées, sorties, registres détruits/préservés, flags modifiés, mémoire utilisée, profondeur de pile, timing critique et hypothèses hardware.

Q438 — EN: Entretien — How should an Assembly routine be documented?

Answer EN: State inputs, outputs, clobbered/preserved registers, modified flags, memory used, stack depth, critical timing, and hardware assumptions.

Q439 — FR : Debug — Comment documenter une routine Assembly proprement ?

Réponse FR : Indiquer entrées, sorties, registres détruits/préservés, flags modifiés, mémoire utilisée, profondeur de pile, timing critique et hypothèses hardware.

Q439 — EN: Debug — How should an Assembly routine be documented?

Answer EN: State inputs, outputs, clobbered/preserved registers, modified flags, memory used, stack depth, critical timing, and hardware assumptions.

Q440 — FR : Code review — Comment documenter une routine Assembly proprement ?

Réponse FR : Indiquer entrées, sorties, registres détruits/préservés, flags modifiés, mémoire utilisée, profondeur de pile, timing critique et hypothèses hardware.

Q440 — EN: Code review — How should an Assembly routine be documented?

Answer EN: State inputs, outputs, clobbered/preserved registers, modified flags, memory used, stack depth, critical timing, and hardware assumptions.

Q441 — FR : Pourquoi un listing assembleur est-il essentiel ?

Réponse FR : Il relie labels, adresses, opcodes et tailles réelles. Il permet de vérifier vectors, pages AJMP/ACALL, placement des tables et taille du firmware.

Q441 — EN: Why is an assembler listing essential?

Answer EN: It connects labels, addresses, opcodes, and actual sizes. It helps verify vectors, AJMP/ACALL pages, table placement, and firmware size.

Q442 — FR : Entretien — Pourquoi un listing assembleur est-il essentiel ?

Réponse FR : Il relie labels, adresses, opcodes et tailles réelles. Il permet de vérifier vectors, pages AJMP/ACALL, placement des tables et taille du firmware.

Q442 — EN: Entretien — Why is an assembler listing essential?

Answer EN: It connects labels, addresses, opcodes, and actual sizes. It helps verify vectors, AJMP/ACALL pages, table placement, and firmware size.

Q443 — FR : Debug — Pourquoi un listing assembleur est-il essentiel ?

Réponse FR : Il relie labels, adresses, opcodes et tailles réelles. Il permet de vérifier vectors, pages AJMP/ACALL, placement des tables et taille du firmware.

Q443 — EN: Debug — Why is an assembler listing essential?

Answer EN: It connects labels, addresses, opcodes, and actual sizes. It helps verify vectors, AJMP/ACALL pages, table placement, and firmware size.

Q444 — FR : Code review — Pourquoi un listing assembleur est-il essentiel ?

CLASSICAL 8051-01 | Architecture & Memory | FR / EN

© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.

Page 63 From 76

[GitHub for more info](#)



Réponse FR : Il relie labels, adresses, opcodes et tailles réelles. Il permet de vérifier vectors, pages AJMP/ACALL, placement des tables et taille du firmware.

Q444 — EN: Code review — Why is an assembler listing essential?

Answer EN: It connects labels, addresses, opcodes, and actual sizes. It helps verify vectors, AJMP/ACALL pages, table placement, and firmware size.

8. Bridge to NXP P89LPC9xx / Pont vers NXP P89LPC9xx

Q445 — FR : En quoi P89LPC915/916/917 diffèrent-ils d'un 80C51 historique ?

Réponse FR : Ils conservent le modèle 80C51 mais utilisent un cœur accéléré two-clock et ajoutent Flash, ADC et autres fonctions selon le modèle. Le code Assembly doit donc séparer instructions du cœur et SFR périphériques NXP.

Q445 — EN: How do P89LPC915/916/917 differ from a historical 80C51?

Answer EN: They retain the 80C51 programming model but use an accelerated two-clock core and add Flash, ADC, and other functions depending on device. Assembly must therefore separate core instructions from NXP peripheral SFRs.

Q446 — FR : Architecture — En quoi P89LPC915/916/917 diffèrent-ils d'un 80C51 historique ?

Réponse FR : Ils conservent le modèle 80C51 mais utilisent un cœur accéléré two-clock et ajoutent Flash, ADC et autres fonctions selon le modèle. Le code Assembly doit donc séparer instructions du cœur et SFR périphériques NXP.

Q446 — EN: Architecture — How do P89LPC915/916/917 differ from a historical 80C51?

Answer EN: They retain the 80C51 programming model but use an accelerated two-clock core and add Flash, ADC, and other functions depending on device. Assembly must therefore separate core instructions from NXP peripheral SFRs.

Q447 — FR : Interview — En quoi P89LPC915/916/917 diffèrent-ils d'un 80C51 historique ?

Réponse FR : Ils conservent le modèle 80C51 mais utilisent un cœur accéléré two-clock et ajoutent Flash, ADC et autres fonctions selon le modèle. Le code Assembly doit donc séparer instructions du cœur et SFR périphériques NXP.

Q447 — EN: Interview — How do P89LPC915/916/917 differ from a historical 80C51?

Answer EN: They retain the 80C51 programming model but use an accelerated two-clock core and add Flash, ADC, and other functions depending on device. Assembly must therefore separate core instructions from NXP peripheral SFRs.

Q448 — FR : Migration — En quoi P89LPC915/916/917 diffèrent-ils d'un 80C51 historique ?

Réponse FR : Ils conservent le modèle 80C51 mais utilisent un cœur accéléré two-clock et ajoutent Flash, ADC et autres fonctions selon le modèle. Le code Assembly doit donc séparer instructions du cœur et SFR périphériques NXP.

Q448 — EN: Migration — How do P89LPC915/916/917 differ from a historical 80C51?

Answer EN: They retain the 80C51 programming model but use an accelerated two-clock core and add Flash, ADC, and other functions depending on device. Assembly must therefore separate core instructions from NXP peripheral SFRs.

Q449 — FR : Quels espaces mémoire retrouve-t-on sur P89LPC9151/9161/9171 ?

Réponse FR : NXP documente DATA, IDATA, SFR, XDATA et CODE; le composant implémente notamment 2 KiB de CODE on-chip et 256 octets d'IDATA, avec des extensions SFR/XDATA selon le modèle.

Q449 — EN: Which memory spaces are documented for P89LPC9151/9161/9171?

Answer EN: NXP documents DATA, IDATA, SFR, XDATA, and CODE; the device implements 2 KiB on-chip CODE and 256 bytes of IDATA, with device-specific SFR/XDATA extensions.

Q450 — FR : Architecture — Quels espaces mémoire retrouve-t-on sur P89LPC9151/9161/9171 ?

Réponse FR : NXP documente DATA, IDATA, SFR, XDATA et CODE; le composant implémente notamment 2 KiB de CODE on-chip et 256



octets d'IDATA, avec des extensions SFR/XDATA selon le modèle.

Q450 — EN: Architecture — Which memory spaces are documented for P89LPC9151/9161/9171?

Answer EN: NXP documents DATA, IDATA, SFR, XDATA, and CODE; the device implements 2 KiB on-chip CODE and 256 bytes of IDATA, with device-specific SFR/XDATA extensions.

Q451 — FR : Interview — Quels espaces mémoire retrouve-t-on sur P89LPC9151/9161/9171 ?

Réponse FR : NXP documente DATA, IDATA, SFR, XDATA et CODE; le composant implémente notamment 2 KiB de CODE on-chip et 256 octets d'IDATA, avec des extensions SFR/XDATA selon le modèle.

Q451 — EN: Interview — Which memory spaces are documented for P89LPC9151/9161/9171?

Answer EN: NXP documents DATA, IDATA, SFR, XDATA, and CODE; the device implements 2 KiB on-chip CODE and 256 bytes of IDATA, with device-specific SFR/XDATA extensions.

Q452 — FR : Migration — Quels espaces mémoire retrouve-t-on sur P89LPC9151/9161/9171 ?

Réponse FR : NXP documente DATA, IDATA, SFR, XDATA et CODE; le composant implémente notamment 2 KiB de CODE on-chip et 256 octets d'IDATA, avec des extensions SFR/XDATA selon le modèle.

Q452 — EN: Migration — Which memory spaces are documented for P89LPC9151/9161/9171?

Answer EN: NXP documents DATA, IDATA, SFR, XDATA, and CODE; the device implements 2 KiB on-chip CODE and 256 bytes of IDATA, with device-specific SFR/XDATA extensions.

Q453 — FR : Pourquoi ne pas apprendre l'ADC dans le volume 01 ?

Réponse FR : Parce que l'ADC n'appartient pas au jeu d'instructions classique 8051. Il est une périphérie du dérivé. Nous établissons d'abord le CPU et l'Assembly; le volume ADC utilisera ensuite les SFR exacts du P89LPC ciblé.

Q453 — EN: Why not teach the ADC in volume 01?

Answer EN: Because the ADC is not part of the classic 8051 instruction set. It is derivative-specific peripheral hardware. We first establish CPU/Assembly fundamentals; the ADC volume will then use the exact SFRs of the selected P89LPC device.

Q454 — FR : Architecture — Pourquoi ne pas apprendre l'ADC dans le volume 01 ?

Réponse FR : Parce que l'ADC n'appartient pas au jeu d'instructions classique 8051. Il est une périphérie du dérivé. Nous établissons d'abord le CPU et l'Assembly; le volume ADC utilisera ensuite les SFR exacts du P89LPC ciblé.

Q454 — EN: Architecture — Why not teach the ADC in volume 01?

Answer EN: Because the ADC is not part of the classic 8051 instruction set. It is derivative-specific peripheral hardware. We first establish CPU/Assembly fundamentals; the ADC volume will then use the exact SFRs of the selected P89LPC device.

Q455 — FR : Interview — Pourquoi ne pas apprendre l'ADC dans le volume 01 ?

Réponse FR : Parce que l'ADC n'appartient pas au jeu d'instructions classique 8051. Il est une périphérie du dérivé. Nous établissons d'abord le CPU et l'Assembly; le volume ADC utilisera ensuite les SFR exacts du P89LPC ciblé.

Q455 — EN: Interview — Why not teach the ADC in volume 01?

Answer EN: Because the ADC is not part of the classic 8051 instruction set. It is derivative-specific peripheral hardware. We first establish CPU/Assembly fundamentals; the ADC volume will then use the exact SFRs of the selected P89LPC device.

Q456 — FR : Migration — Pourquoi ne pas apprendre l'ADC dans le volume 01 ?

Réponse FR : Parce que l'ADC n'appartient pas au jeu d'instructions classique 8051. Il est une périphérie du dérivé. Nous établissons d'abord le CPU et l'Assembly; le volume ADC utilisera ensuite les SFR exacts du P89LPC ciblé.

Q456 — EN: Migration — Why not teach the ADC in volume 01?

Answer EN: Because the ADC is not part of the classic 8051 instruction set. It is derivative-specific peripheral hardware. We first establish CPU/Assembly fundamentals; the ADC volume will then use the exact SFRs of the selected P89LPC device.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

9. Assembly code appendix / Annexe de code Assembly

Minimal reset skeleton

```
ORG 0000h
LJMP START
```

```
START:  MOV SP,#2Fh
        CLR A
MAIN:   SJMP MAIN
```

```
END
```

Internal RAM copy loop

```
MOV R0,#30h
MOV R1,#40h
MOV R7,#16
COPY_LOOP: MOV A,@R0
            MOV @R1,A
            INC R0
            INC R1
            DJNZ R7,COPY_LOOP
```

16-bit addition

```
; R5:R4 + R7:R6 -> R1:R0
MOV A,R4
ADD A,R6
MOV R0,A
MOV A,R5
ADDC A,R7
MOV R1,A
```

CODE lookup table

```
MOV DPTR,#TABLE
MOV A,R0
MOVC A,@A+DPTR
; A = TABLE[R0]
```

```
TABLE:  DB 10h,20h,30h,40h
```

Bit flag consume

```
FLAG0 EQU 00h ; bit address 00h = RAM 20h.0
JBC FLAG0,HANDLE
SJMP CONTINUE
HANDLE: ; event handler
CONTINUE:
```

Subroutine register preservation

```
PROCESS: PUSH ACC
        PUSH PSW
        ; routine body
        POP PSW
        POP ACC
        RET
```

Multi-byte left shift

```
CLR C
MOV A,R0
RLC A
MOV R0,A
MOV A,R1
RLC A
MOV R1,A
```

XDATA access

```
MOV DPTR,#1234h
MOV A,#55h
MOVX @DPTR,A
MOVX A,@DPTR
```

10. Mastery checklist / Liste de maîtrise

- ☐ Explain A, B, PSW, DPTR, PC, SP and R0-R7 without confusing CPU registers with peripheral SFRs.
- ☐ Draw the classic DATA/IDATA/SFR/XDATA/CODE memory model.
- ☐ Explain why address 80h can mean an SFR under direct addressing and upper IDATA on derivatives under indirect addressing.
- ☐ Use all major addressing modes and select MOV, MOVX or MOVC correctly.
- ☐ Plan register banks, variables and stack so they do not overlap.
- ☐ Explain CY, AC, OV and P and their role in arithmetic.
- ☐ Write multi-byte add/subtract/shift routines using CY.
- ☐ Use bit-addressable RAM/SFR operations effectively.
- ☐ Choose SJMP/AJMP/LJMP and ACALL/LCALL based on reach and placement.
- ☐ Use CALL/RET and interrupt-oriented RETI correctly.
- ☐ Read lookup tables from CODE with MOVC.
- ☐ Read assembler listings and verify addresses/opcodes/timing.
- ☐ Separate classic 8051 behavior from NXP P89LPC9xx peripheral extensions.
- ☐ Write and review 8051 Assembly without relying on C abstractions.

References / Références techniques

- NXP Semiconductors, UM10107 — P89LPC915/916/917 User manual, Rev. 02, 12 Nov 2009.
- NXP Semiconductors, P89LPC915/916/917 data sheet, Rev. 1.0, 16 Dec 2009.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com*

- NXP Semiconductors, UM10344 — P89LPC9151/9161/9171 User manual.
- Device-specific data sheet/user manual must be checked before using derivative-specific SFR addresses or timing.

11. Focused syntax drills / Exercices ciblés de syntaxe

Q457 — FR : Que signifie la forme ``MOV A,#data`` ?

Réponse FR : Cette forme charge une constante immédiate dans A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q457 — EN: What does the form ``MOV A,#data`` mean?

Answer EN: This form loads an immediate constant into A. Verify modified flags and exact operand legality for the assembler being used.

Q458 — FR : Quel test minimal utiliser pour vérifier ``MOV A,#data`` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois ``MOV A,#data``, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q458 — EN: What minimal test should verify ``MOV A,#data``?

Answer EN: Initialize operands to known values, execute ``MOV A,#data`` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q459 — FR : Que signifie la forme ``MOV A,direct`` ?

Réponse FR : Cette forme lit un octet direct DATA/SFR dans A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q459 — EN: What does the form ``MOV A,direct`` mean?

Answer EN: This form reads a direct DATA/SFR byte into A. Verify modified flags and exact operand legality for the assembler being used.

Q460 — FR : Quel test minimal utiliser pour vérifier ``MOV A,direct`` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois ``MOV A,direct``, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q460 — EN: What minimal test should verify ``MOV A,direct``?

Answer EN: Initialize operands to known values, execute ``MOV A,direct`` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q461 — FR : Que signifie la forme ``MOV direct,A`` ?

Réponse FR : Cette forme écrit A vers une adresse directe. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q461 — EN: What does the form ``MOV direct,A`` mean?

Answer EN: This form writes A to a direct address. Verify modified flags and exact operand legality for the assembler being used.

Q462 — FR : Quel test minimal utiliser pour vérifier ``MOV direct,A`` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois ``MOV direct,A``, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q462 — EN: What minimal test should verify ``MOV direct,A``?

Answer EN: Initialize operands to known values, execute ``MOV direct,A`` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q463 — FR : Que signifie la forme ``MOV A,@R0`` ?

CLASSICAL 8051-01 | Architecture & Memory | FR / EN

© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.

Page 68 From 76

[GitHub for more info](#)



Réponse FR : Cette forme lit la RAM interne indirectement via R0. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q463 — EN: What does the form `MOV A,@R0` mean?

Answer EN: This form reads internal RAM indirectly through R0. Verify modified flags and exact operand legality for the assembler being used.

Q464 — FR : Quel test minimal utiliser pour vérifier `MOV A,@R0` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `MOV A,@R0`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q464 — EN: What minimal test should verify `MOV A,@R0`?

Answer EN: Initialize operands to known values, execute `MOV A,@R0` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q465 — FR : Que signifie la forme `MOV @R1,A` ?

Réponse FR : Cette forme écrit A indirectement via R1. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q465 — EN: What does the form `MOV @R1,A` mean?

Answer EN: This form writes A indirectly through R1. Verify modified flags and exact operand legality for the assembler being used.

Q466 — FR : Quel test minimal utiliser pour vérifier `MOV @R1,A` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `MOV @R1,A`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q466 — EN: What minimal test should verify `MOV @R1,A`?

Answer EN: Initialize operands to known values, execute `MOV @R1,A` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q467 — FR : Que signifie la forme `MOV DPTR,#data16` ?

Réponse FR : Cette forme charge une constante 16 bits dans DPTR. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q467 — EN: What does the form `MOV DPTR,#data16` mean?

Answer EN: This form loads a 16-bit constant into DPTR. Verify modified flags and exact operand legality for the assembler being used.

Q468 — FR : Quel test minimal utiliser pour vérifier `MOV DPTR,#data16` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `MOV DPTR,#data16`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q468 — EN: What minimal test should verify `MOV DPTR,#data16`?

Answer EN: Initialize operands to known values, execute `MOV DPTR,#data16` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q469 — FR : Que signifie la forme `MOVX A,@DPTR` ?

Réponse FR : Cette forme lit XDATA via DPTR. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q469 — EN: What does the form `MOVX A,@DPTR` mean?

Answer EN: This form reads XDATA through DPTR. Verify modified flags and exact operand legality for the assembler being used.

Q470 — FR : Quel test minimal utiliser pour vérifier `MOVX A,@DPTR` ?



Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `MOVX A,@DPTR`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q470 — EN: What minimal test should verify `MOVX A,@DPTR`?

Answer EN: Initialize operands to known values, execute `MOVX A,@DPTR` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q471 — FR : Que signifie la forme `MOVX @DPTR,A` ?

Réponse FR : Cette forme écrit XDATA via DPTR. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q471 — EN: What does the form `MOVX @DPTR,A` mean?

Answer EN: This form writes XDATA through DPTR. Verify modified flags and exact operand legality for the assembler being used.

Q472 — FR : Quel test minimal utiliser pour vérifier `MOVX @DPTR,A` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `MOVX @DPTR,A`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q472 — EN: What minimal test should verify `MOVX @DPTR,A`?

Answer EN: Initialize operands to known values, execute `MOVX @DPTR,A` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q473 — FR : Que signifie la forme `MOVC A,@A+DPTR` ?

Réponse FR : Cette forme lit CODE avec index A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q473 — EN: What does the form `MOVC A,@A+DPTR` mean?

Answer EN: This form reads CODE using A as an index. Verify modified flags and exact operand legality for the assembler being used.

Q474 — FR : Quel test minimal utiliser pour vérifier `MOVC A,@A+DPTR` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `MOVC A,@A+DPTR`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q474 — EN: What minimal test should verify `MOVC A,@A+DPTR`?

Answer EN: Initialize operands to known values, execute `MOVC A,@A+DPTR` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q475 — FR : Que signifie la forme `ADD A,#data` ?

Réponse FR : Cette forme addition immédiate à A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q475 — EN: What does the form `ADD A,#data` mean?

Answer EN: This form adds an immediate value to A. Verify modified flags and exact operand legality for the assembler being used.

Q476 — FR : Quel test minimal utiliser pour vérifier `ADD A,#data` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `ADD A,#data`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q476 — EN: What minimal test should verify `ADD A,#data`?

Answer EN: Initialize operands to known values, execute `ADD A,#data` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q477 — FR : Que signifie la forme `ADDC A,Rn` ?

Réponse FR : Cette forme additionne Rn et CY à A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q477 — EN: What does the form `ADDC A,Rn` mean?

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Answer EN: This form adds Rn and CY to A. Verify modified flags and exact operand legality for the assembler being used.

Q478 — FR : Quel test minimal utiliser pour vérifier `ADDC A,Rn` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `ADDC A,Rn`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q478 — EN: What minimal test should verify `ADDC A,Rn`?

Answer EN: Initialize operands to known values, execute `ADDC A,Rn` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q479 — FR : Que signifie la forme `SUBB A,direct` ?

Réponse FR : Cette forme soustrait direct et CY de A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q479 — EN: What does the form `SUBB A,direct` mean?

Answer EN: This form subtracts direct byte and CY from A. Verify modified flags and exact operand legality for the assembler being used.

Q480 — FR : Quel test minimal utiliser pour vérifier `SUBB A,direct` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `SUBB A,direct`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q480 — EN: What minimal test should verify `SUBB A,direct`?

Answer EN: Initialize operands to known values, execute `SUBB A,direct` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q481 — FR : Que signifie la forme `ANL A,#mask` ?

Réponse FR : Cette forme masque des bits de A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q481 — EN: What does the form `ANL A,#mask` mean?

Answer EN: This form masks bits in A. Verify modified flags and exact operand legality for the assembler being used.

Q482 — FR : Quel test minimal utiliser pour vérifier `ANL A,#mask` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `ANL A,#mask`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q482 — EN: What minimal test should verify `ANL A,#mask`?

Answer EN: Initialize operands to known values, execute `ANL A,#mask` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q483 — FR : Que signifie la forme `ORL direct,#mask` ?

Réponse FR : Cette forme met certains bits d'un octet direct. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q483 — EN: What does the form `ORL direct,#mask` mean?

Answer EN: This form sets selected bits in a direct byte. Verify modified flags and exact operand legality for the assembler being used.

Q484 — FR : Quel test minimal utiliser pour vérifier `ORL direct,#mask` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `ORL direct,#mask`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q484 — EN: What minimal test should verify `ORL direct,#mask`?

Answer EN: Initialize operands to known values, execute `ORL direct,#mask` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

**Q485 — FR : Que signifie la forme `XRL A,#mask` ?**

Réponse FR : Cette forme bascule certains bits de A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q485 — EN: What does the form `XRL A,#mask` mean?

Answer EN: This form toggles selected bits in A. Verify modified flags and exact operand legality for the assembler being used.

Q486 — FR : Quel test minimal utiliser pour vérifier `XRL A,#mask` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `XRL A,#mask`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q486 — EN: What minimal test should verify `XRL A,#mask`?

Answer EN: Initialize operands to known values, execute `XRL A,#mask` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q487 — FR : Que signifie la forme `CLR C` ?

Réponse FR : Cette forme efface CY. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q487 — EN: What does the form `CLR C` mean?

Answer EN: This form clears CY. Verify modified flags and exact operand legality for the assembler being used.

Q488 — FR : Quel test minimal utiliser pour vérifier `CLR C` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `CLR C`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q488 — EN: What minimal test should verify `CLR C`?

Answer EN: Initialize operands to known values, execute `CLR C` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q489 — FR : Que signifie la forme `SETB C` ?

Réponse FR : Cette forme met CY à 1. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q489 — EN: What does the form `SETB C` mean?

Answer EN: This form sets CY. Verify modified flags and exact operand legality for the assembler being used.

Q490 — FR : Quel test minimal utiliser pour vérifier `SETB C` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `SETB C`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q490 — EN: What minimal test should verify `SETB C`?

Answer EN: Initialize operands to known values, execute `SETB C` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q491 — FR : Que signifie la forme `CPL C` ?

Réponse FR : Cette forme inverse CY. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q491 — EN: What does the form `CPL C` mean?

Answer EN: This form complements CY. Verify modified flags and exact operand legality for the assembler being used.

Q492 — FR : Quel test minimal utiliser pour vérifier `CPL C` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `CPL C`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q492 — EN: What minimal test should verify `CPL C`?

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.***Email: ahds@engineer.com**

Answer EN: Initialize operands to known values, execute `CPL C` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q493 — FR : Que signifie la forme `CLR bit` ?

Réponse FR : Cette forme efface un bit adressable. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q493 — EN: What does the form `CLR bit` mean?

Answer EN: This form clears an addressable bit. Verify modified flags and exact operand legality for the assembler being used.

Q494 — FR : Quel test minimal utiliser pour vérifier `CLR bit` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `CLR bit`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q494 — EN: What minimal test should verify `CLR bit`?

Answer EN: Initialize operands to known values, execute `CLR bit` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q495 — FR : Que signifie la forme `SETB bit` ?

Réponse FR : Cette forme met un bit adressable à 1. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q495 — EN: What does the form `SETB bit` mean?

Answer EN: This form sets an addressable bit. Verify modified flags and exact operand legality for the assembler being used.

Q496 — FR : Quel test minimal utiliser pour vérifier `SETB bit` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `SETB bit`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q496 — EN: What minimal test should verify `SETB bit`?

Answer EN: Initialize operands to known values, execute `SETB bit` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q497 — FR : Que signifie la forme `JB bit,rel` ?

Réponse FR : Cette forme branche si bit=1. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q497 — EN: What does the form `JB bit,rel` mean?

Answer EN: This form branches if bit=1. Verify modified flags and exact operand legality for the assembler being used.

Q498 — FR : Quel test minimal utiliser pour vérifier `JB bit,rel` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `JB bit,rel`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q498 — EN: What minimal test should verify `JB bit,rel`?

Answer EN: Initialize operands to known values, execute `JB bit,rel` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q499 — FR : Que signifie la forme `JNB bit,rel` ?

Réponse FR : Cette forme branche si bit=0. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q499 — EN: What does the form `JNB bit,rel` mean?

Answer EN: This form branches if bit=0. Verify modified flags and exact operand legality for the assembler being used.

Q500 — FR : Quel test minimal utiliser pour vérifier `JNB bit,rel` ?



Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `JNB bit,rel`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q500 — EN: What minimal test should verify `JNB bit,rel`?

Answer EN: Initialize operands to known values, execute `JNB bit,rel` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q501 — FR : Que signifie la forme `JBC bit,rel` ?

Réponse FR : Cette forme branche si bit=1 puis l'efface. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q501 — EN: What does the form `JBC bit,rel` mean?

Answer EN: This form branches if bit=1 then clears it. Verify modified flags and exact operand legality for the assembler being used.

Q502 — FR : Quel test minimal utiliser pour vérifier `JBC bit,rel` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `JBC bit,rel`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q502 — EN: What minimal test should verify `JBC bit,rel`?

Answer EN: Initialize operands to known values, execute `JBC bit,rel` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q503 — FR : Que signifie la forme `CJNE A,#data,rel` ?

Réponse FR : Cette forme compare A à une constante et branche si différent. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q503 — EN: What does the form `CJNE A,#data,rel` mean?

Answer EN: This form compares A with a constant and branches if unequal. Verify modified flags and exact operand legality for the assembler being used.

Q504 — FR : Quel test minimal utiliser pour vérifier `CJNE A,#data,rel` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `CJNE A,#data,rel`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q504 — EN: What minimal test should verify `CJNE A,#data,rel`?

Answer EN: Initialize operands to known values, execute `CJNE A,#data,rel` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q505 — FR : Que signifie la forme `DJNZ Rn,rel` ?

Réponse FR : Cette forme décrémente Rn et boucle si non nul. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q505 — EN: What does the form `DJNZ Rn,rel` mean?

Answer EN: This form decrements Rn and loops if nonzero. Verify modified flags and exact operand legality for the assembler being used.

Q506 — FR : Quel test minimal utiliser pour vérifier `DJNZ Rn,rel` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `DJNZ Rn,rel`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q506 — EN: What minimal test should verify `DJNZ Rn,rel`?

Answer EN: Initialize operands to known values, execute `DJNZ Rn,rel` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q507 — FR : Que signifie la forme `PUSH direct` ?



Réponse FR : Cette forme empile un octet direct. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q507 — EN: What does the form `PUSH direct` mean?

Answer EN: This form pushes a direct byte. Verify modified flags and exact operand legality for the assembler being used.

Q508 — FR : Quel test minimal utiliser pour vérifier `PUSH direct` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `PUSH direct`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q508 — EN: What minimal test should verify `PUSH direct`?

Answer EN: Initialize operands to known values, execute `PUSH direct` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q509 — FR : Que signifie la forme `POP direct` ?

Réponse FR : Cette forme dépile vers un octet direct. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q509 — EN: What does the form `POP direct` mean?

Answer EN: This form pops into a direct byte. Verify modified flags and exact operand legality for the assembler being used.

Q510 — FR : Quel test minimal utiliser pour vérifier `POP direct` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `POP direct`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q510 — EN: What minimal test should verify `POP direct`?

Answer EN: Initialize operands to known values, execute `POP direct` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q511 — FR : Que signifie la forme `RLC A` ?

Réponse FR : Cette forme rotation gauche à travers CY. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q511 — EN: What does the form `RLC A` mean?

Answer EN: This form rotates A left through CY. Verify modified flags and exact operand legality for the assembler being used.

Q512 — FR : Quel test minimal utiliser pour vérifier `RLC A` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `RLC A`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q512 — EN: What minimal test should verify `RLC A`?

Answer EN: Initialize operands to known values, execute `RLC A` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q513 — FR : Que signifie la forme `RRC A` ?

Réponse FR : Cette forme rotation droite à travers CY. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q513 — EN: What does the form `RRC A` mean?

Answer EN: This form rotates A right through CY. Verify modified flags and exact operand legality for the assembler being used.

Q514 — FR : Quel test minimal utiliser pour vérifier `RRC A` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `RRC A`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

**AHDS**Annissoff Hardware Development System
*Design once. Reuse forever.**Email: ahds@engineer.com***Q514 — EN: What minimal test should verify `RRC A`?**

Answer EN: Initialize operands to known values, execute `RRC A` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Q515 — FR : Que signifie la forme `SWAP A` ?

Réponse FR : Cette forme échange les deux nibbles de A. Vérifiez les flags modifiés et la légalité exacte de l'opérande pour l'assembleur utilisé.

Q515 — EN: What does the form `SWAP A` mean?

Answer EN: This form swaps the two nibbles of A. Verify modified flags and exact operand legality for the assembler being used.

Q516 — FR : Quel test minimal utiliser pour vérifier `SWAP A` ?

Réponse FR : Initialiser les opérandes à des valeurs connues, exécuter une seule fois `SWAP A`, puis observer A/CY/PSW et la mémoire concernée. Inclure ensuite une valeur limite.

Q516 — EN: What minimal test should verify `SWAP A`?

Answer EN: Initialize operands to known values, execute `SWAP A` once, then inspect A/CY/PSW and affected memory. Then include a boundary value.

Total bilingual Q&A pairs: 516